MANIPAL
ACADEMY of HIGHER EDUCATION
(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES
(A Constituent unit of MAHE, Manipal)

A PROJECT REPORT ON

AUTONOMOUS FIRMWARE FAULT LOCALIZATION USING RUNTIME EXECUTION SIGNATURES

Reg. Number	Name	Branch
261100620002	ABHAY KIRAN M	EMBEDDED SYSTEMS
261100620006	ABHISHEK SATHYENDRAN	EMBEDDED SYSTEMS
261100620024	ARFAN SAYYAD	EMBEDDED SYSTEMS

Under the guidance of

RAVIKALA KAMATH

Assistant Professor – Senior Scale,
Manipal School of Information Sciences,
MAHE, MANIPAL

November 2026

MANIPAL SCHOOL OF INFORMATION SCIENCES
MANIPAL
(A constituent unit of MAHE, Manipal)

ACKNOWLEDGEMENTS

We take this opportunity to thank everyone who supported us while we carried out this project, "Autonomous Firmware Fault Localization using Runtime Execution Signatures".

We are sincerely grateful to our guide, **Ms. Ravikala Kamath**, Assistant Professor – Senior Scale, Manipal School of Information Sciences, for her guidance motivation and insightful suggestions throughout the preparation of this project.

We are also grateful to **Dr. Keerthana Prasad**, Professor and Director, Manipal School of Information Sciences, MAHE, Manipal, for her constant encouragement and for providing the necessary facilities in the institute.

We thank the faculty of the Manipal School of Information Sciences, MAHE, Manipal, for the laboratory facilities and for the coursework in embedded systems and real-time operating systems that gave us the background to attempt this work.

Finally, we would like to acknowledge the contributions of the authors of the published works cited in the references, whose research has been instrumental in shaping this study.

ABHAY KIRAN M
ABHISHEK SATHYENDRAN
ARFAN SAYYAD

ABSTRACT

Faults in embedded firmware are often difficult to identify because they do not always produce clear warning messages. Problems such as task failures, stack overflow, excessive interrupts, incorrect peripheral configurations, and memory access errors can cause unexpected system behaviour or complete system failure. Identifying these faults generally requires external debugging tools. This project presents an autonomous firmware fault localisation system that detects, identifies, and handles faults directly on the microcontroller without requiring external debugging equipment.

The proposed system was developed using an STM32 NUCLEO-F446RE board with an ARM Cortex-M4 processor operating at 84 MHz and running FreeRTOS. It monitors eight parameters: task execution sequence, interrupt timing, execution time, stack usage, peripheral configuration, watchdog events, memory access errors, and communication errors. The framework uses internal resources such as the DWT cycle counter, FreeRTOS trace hooks, an 8 KB RAM ring buffer, Memory Protection Unit (MPU), system fault registers, watchdog, and flash memory. Every 100 ms, execution data is compared with previously recorded normal behaviour. When an abnormal condition is detected, the system identifies the likely faulty component and assigns a confidence score between 30% and 99%. Recovery actions include restarting a task, reinitialising a peripheral, or resetting the microcontroller. A 164-byte fault report containing the last 16 execution events is stored in flash memory.

Testing was performed using a real-time sensor application consisting of a 1 kHz ADC data acquisition system with DMA, a four-stage processing pipeline running every 10 ms, and a telemetry task running every 100 ms. Ten fault types were introduced through a UART command interface, and 18 test cases were conducted. All 15 fault events were successfully detected, including register corruption and unexpected UART communication errors. Of these, 14 were correctly identified on the first attempt. Runtime faults were detected within 31–172 ms, with a median detection time of 72 ms, while memory protection faults were detected within 9–239 µs. No false alarms occurred during normal operation.

The framework introduced approximately 15 KB of RAM usage, along with an additional 8 KB trace buffer. Comparison of rule-based and statistical detection methods showed similar accuracy for actual faults, although the statistical method produced more false alarms during event-driven task execution.

The results demonstrate that embedded firmware faults can be detected, localised, and handled directly by the microcontroller, providing a lightweight solution for improving firmware reliability without external debugging equipment.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION..... 1
1.1 PROBLEM STATEMENT..... 3
1.2 OBJECTIVES..... 3
1.3 SCOPE OF THE PROJECT..... 4
CHAPTER 2 LITERATURE REVIEW..... 5
2.1 COMPARISON OF RELATED WORK7
CHAPTER 3 SYSTEM REQUIREMENTS AND SPECIFICATIONS..... 8
3.1 FUNCTIONAL REQUIREMENTS..... 8
3.2 NON-FUNCTIONAL REQUIREMENTS..... 8
3.3 HARDWARE SPECIFICATIONS..... 9
3.4 SOFTWARE SPECIFICATIONS..... 10
3.5 SIGNALS TO BE OBSERVED..... 10
CHAPTER 4 SYSTEM ARCHITECTURE AND DESIGN..... 11
4.1 DESIGN OVERVIEW..... 11
4.2 SOFTWARE ARCHITECTURE..... 11
4.3 TASK AND INTERRUPT MODEL..... 13
4.4 SHARED INTERFACES..... 14
4.5 MEMORY LAYOUT..... 14
4.6 DESIGN DECISIONS..... 15
CHAPTER 5 THE MONITORED APPLICATION..... 17
5.1 PURPOSE OF THE APPLICATION..... 17
5.2 HARDWARE-TIMED ACQUISITION..... 17
5.3 THE SENSOR TASK..... 18
5.4 TELEMETRY AND INPUTS..... 18
5.5 COMMAND INTERFACE..... 18
CHAPTER 6 ON-CHIP CAPTURE USING CORTEX-M4 FEATURES..... 20
6.1 ARMV7-M DEBUG AND FAULT ARCHITECTURE..... 20
6.2 TIMESTAMPS FROM THE DWT CYCLE COUNTER..... 20
6.3 TRACE HOOKS..... 21
6.4 EVENT RING BUFFER..... 22
6.5 THE 100 MS SIGNATURE WINDOW..... 23
6.6 MEMORY PROTECTION UNIT..... 24
6.7 FAULT EXCEPTIONS AND THE TRAP PATH..... 24
6.8 RESET CAUSE AND RESET-SURVIVING RAM..... 25
6.9 PERIPHERAL AND COMMUNICATION ERRORS..... 26
CHAPTER 7 BASELINE LEARNING AND DETECTION..... 27
7.1 THE MONITOR TASK..... 27
7.2 BASELINE LEARNING..... 27
7.3 THRESHOLDS..... 28
7.4 DETECTION RULES..... 28
7.5 STATISTICAL LOCALIZER..... 29
CHAPTER 8 LOCALIZATION, CONFIDENCE AND RECOVERY..... 30

8.1 LOCALIZATION PIPELINE..... 30
8.2 CONFIDENCE SCORING..... 30
8.3 RECOVERY ENGINE..... 31
8.4 WATCHDOG DESIGN..... 32
8.5 FAULT REPORT AND FLASH BLACK BOX..... 32
CHAPTER 9 FAULT INJECTION FRAMEWORK..... 34
9.1 PROPOSED ARCHITECTURE..... 34
9.2 FAULT CATALOGUE..... 35
9.3 TASK FAULTS..... 35
9.4 HARDWARE FAULTS..... 36
CHAPTER 10 REAL-TIME CLOCK..... 37
10.1 DESIGN..... 37
CHAPTER 11 TESTING METHODOLOGY..... 39
11.1 PROCEDURE..... 39
11.2 TEST CASES..... 41
CHAPTER 12 RESULTS..... 42
12.1 TEST 1: BOOT AND SYSTEM HEALTH..... 42
12.2 TEST 2: BASELINE LEARNING..... 42
12.3 TEST 3: CLEAN RUN (FALSE-ALARM CHECK)..... 43
12.4 TEST 4: APPLICATION INPUT (PA0 AT GND AND 3.3 V)..... 44
12.5 TEST 5: STUCK TASK..... 45
12.6 TEST 6: ADC TRIGGER CORRUPTED..... 45
12.7 TEST 7: SAMPLING TIMER CORRUPTED..... 46
12.8 TEST 8: REGISTER WRITE WITHOUT THE INJECTOR..... 46
12.9 TEST 9: MISSED DEADLINE..... 47
12.10 TEST 10: WRONG EXECUTION SEQUENCE..... 48
12.11 TEST 11: INTERRUPT STORM..... 48
12.12 TEST 12: CORRUPTED COMMAND FRAME..... 49
12.13 TEST 13: WRITE TO PROTECTED MEMORY..... 49
12.14 TEST 14: STACK OVERFLOW..... 50
12.15 TEST 15: TOTAL HANG AND WATCHDOG RESET..... 50
12.16 TEST 16: RULE-BASED VS STATISTICAL LOCALIZER..... 51
12.17 TEST 17: CANCELLING A PENDING FAULT..... 52
12.18 TEST 18: FAULT LOG AFTER THE CAMPAIGN..... 52
12.19 IMPACT ON THE APPLICATION..... 53
12.20 SUMMARY OF MEASURED RESULTS..... 53
CHAPTER 13 DISCUSSION..... 54
13.1 ANSWER TO THE RESEARCH QUESTION..... 54
13.2 WHY HARD EVIDENCE MATTERS..... 54
13.3 TIMING EVIDENCE AND THE LOGARITHMIC SCORE..... 54
13.4 RULE-BASED VERSUS STATISTICAL..... 54
13.5 COMPARISON WITH OTHER APPROACHES..... 55
CHAPTER 14 CHALLENGES AND LIMITATIONS..... 56
14.1 ISSUES FACED AND HOW THEY WERE RESOLVED..... 56

14.2 LIMITATIONS.....56
CHAPTER 15 CONCLUSION AND FUTURE SCOPE.....58
15.1 CONCLUSION.....58
15.2 FUTURE SCOPE.....60
REFERENCES.....61

LIST OF FIGURES

Figure 3.1: Hardware block diagram of the target setup....9
Figure 4.1: Three-stage approach: Capture, Compare, Localize and Recover....11
Figure 4.2: Layered software architecture and the hardware features it uses....12
Figure 4.3: Use of flash and SRAM, including the MPU-protected regions....15
Figure 5.1: Data path of the sensor application from the timer trigger to the alarm LED....17
Figure 6.1: Organisation of the event ring buffer....22
Figure 6.2: Construction of the signature vector in each 100 ms window....23
Figure 6.3: Hardware-stacked exception frame and the registers used to diagnose a fault....25
Figure 7.1: Life cycle of the monitor task....27
Figure 8.1: Localization pipeline from rule hits to the winning candidate....30
Figure 8.2: Confidence as a function of the margin (rule-based) and of S/T (statistical)....31
Figure 8.3: Escalation of recovery for repeated faults on the same component....32
Figure 9.1: Path of a fault from the command to the ground-truth record....34
Figure 10.1: Inputs and outputs of the real-time clock add-on....37
Figure 12.1: Serial output of Test 1....42
Figure 12.2: Serial output of Test 2 (a)....42
Figure 12.3: Serial output of Test 2 (b)....43
Figure 12.4: Serial output of Test 3 (a)....43
Figure 12.5: Serial output of Test 3 (b)....43
Figure 12.6: Serial output of Test 4 (a)....44
Figure 12.7: Serial output of Test 4 (b)....44
Figure 12.8: Serial output of Test 5....45
Figure 12.9: Serial output of Test 6....45
Figure 12.10: Serial output of Test 7....46
Figure 12.11: Serial output of Test 8....46
Figure 12.12: Serial output of Test 9....47
Figure 12.13: Serial output of Test 10....48
Figure 12.14: Serial output of Test 11....48
Figure 12.15: Serial output of Test 12 (a)....49
Figure 12.16: Serial output of Test 12 (b)....49
Figure 12.17: Serial output of Test 13....49
Figure 12.18: Serial output of Test 14....50
Figure 12.19: Serial output of Test 15....50
Figure 12.20: Serial output of Test 16 (a)....51
Figure 12.21: Serial output of Test 16 (b)....51
Figure 12.22: Serial output of Test 17....52
Figure 12.23: Serial output of Test 18....52

LIST OF TABLES

Table 2.1: Comparison of related work with this project.....6
Table 3.1: Hardware used in the project.....9
Table 3.2: Software tools and libraries.....10
Table 3.3: The eight observed signals and how they are captured.....10
Table 4.1: Tasks and interrupts of the system.....13
Table 5.1: Command interface.....19
Table 6.1: Debug and protection features of the Cortex-M4 and how they are used.....20
Table 6.2: DWT registers used.....21
Table 6.3: MPU regions used.....24
Table 7.1: Detection thresholds.....28
Table 7.2: Detection rules and their classes.....29
Table 8.1: Confidence assigned to each type of evidence.....31
Table 8.2: Recovery table.....31
Table 8.3: Contents of a fault report.....33
Table 9.1: Injectable faults.....35
Table 11.1: Test cases of the campaign.....41
Table 12.1: Key results of the test campaign.....53
Table 14.1: Issues faced during the project and their resolution.....56

ABBREVIATIONS

Abbreviation	Full form
ADC	Analog-to-Digital Converter
API	Application Programming Interface
ARM	Advanced RISC Machines
BFAR	BusFault Address Register
CFSR	Configurable Fault Status Register
CMSIS	Common Microcontroller Software Interface Standard
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DEMCR	Debug Exception and Monitor Control Register
DMA	Direct Memory Access
DWT	Data Watchpoint and Trace unit
ETM	Embedded Trace Macrocell
EXC_RETURN	Exception return value held in LR on exception entry
FPB	Flash Patch and Breakpoint unit
FPU	Floating-Point Unit
GPIO	General-Purpose Input/Output
HAL	Hardware Abstraction Layer
HFSR	HardFault Status Register
IRQ	Interrupt Request
ISR	Interrupt Service Routine
ITM	Instrumentation Trace Macrocell
IWDG	Independent Watchdog
LED	Light-Emitting Diode
LSI	Low-Speed Internal oscillator
MCU	Microcontroller Unit
MMFAR	MemManage Fault Address Register
MPU	Memory Protection Unit
MSP	Main Stack Pointer
NVIC	Nested Vectored Interrupt Controller
PC	Program Counter
PRIMASK	Priority Mask register (global interrupt disable)
PSP	Process Stack Pointer
RAM	Random-Access Memory
RCC_CSR	Reset and Clock Control, Control/Status Register
RTOS	Real-Time Operating System
SCB	System Control Block
SRAM	Static Random-Access Memory
SWO	Serial Wire Output
TCB	Task Control Block
UART / USART	Universal (Synchronous/) Asynchronous Receiver/Transmitter
VCP	Virtual COM Port

CHAPTER 1
INTRODUCTION

Microcontrollers are widely used in modern electronic systems to perform monitoring, control, communication, and automation tasks. They are found in applications such as automotive electronics, industrial automation, medical equipment, robotics, power management systems, and smart devices. Most of these systems depend on embedded firmware to control hardware operations and ensure that tasks are executed within their specified time limits. As the complexity of embedded applications increases, ensuring the reliability and continuous operation of firmware becomes increasingly important.

Many modern embedded systems use a Real-Time Operating System (RTOS) to manage multiple tasks with different priorities and execution periods. These tasks communicate with hardware peripherals, process sensor data, control actuators, and handle communication protocols. Interrupt Service Routines (ISRs) are also used to respond to important hardware events. Since these operations are time-sensitive, even a small firmware error can affect task execution, interrupt handling, system timing, and overall performance. Such errors may eventually lead to unexpected behaviour, incorrect outputs, system instability, or complete system failure.

Traditional firmware debugging is generally performed using external debugging tools such as Serial Wire Debug (SWD), breakpoints, hardware trace units, and development environments. These tools are highly effective during development and testing because they allow engineers to examine program execution, inspect memory, monitor registers, and identify errors. However, these methods are not always practical once an embedded device is deployed in the field. Deployed systems may operate continuously for extended periods without direct human supervision or access to external debugging equipment. Consequently, identifying the cause of an unexpected failure becomes difficult, particularly when the fault occurs intermittently or cannot be reproduced under laboratory conditions.

To overcome these limitations, this project proposes a system for autonomous firmware fault localization using runtime execution signatures. The main objective is to develop an intelligent monitoring mechanism that continuously observes firmware execution, identifies abnormal behaviour, and estimates the location and severity of potential faults without requiring an external debugger or host computer.

The proposed approach is based on the concept of Runtime Execution Signatures. In a healthy embedded system, periodic tasks generally follow predictable execution patterns. Their execution time, activation frequency, processing sequence, and completion behaviour remain within an expected range under normal operating conditions. These characteristics can be collected during normal system operation and used to establish a reference execution signature for each task. During runtime, the actual execution behaviour is continuously compared with these reference signatures. Any significant deviation can indicate a possible firmware fault, abnormal execution condition, or timing violation.

The system uses the hardware monitoring capabilities available in ARM Cortex-M microcontrollers to collect execution-related information. The Data Watchpoint and Trace (DWT) unit provides a cycle counter that can be used to measure task execution time with high resolution. Runtime monitoring also considers task activation intervals, execution durations, missed deadlines, watchdog status, and other relevant execution events. These parameters are recorded using a lightweight event-monitoring mechanism designed to minimize the additional processing and memory overhead introduced by the monitoring system.

A lightweight fault-localization mechanism analyses the collected execution signatures to identify abnormal task behaviour. Based on the observed deviations, the system calculates a confidence score for each monitored task, represented as a percentage between 0 and 100. This score indicates the level of confidence that a particular task is associated with the detected abnormal behaviour. By evaluating multiple runtime parameters rather than depending on a single failure indicator, the system aims to provide more useful information about the possible source of a fault.

The proposed system is implemented on an STM32F446RE microcontroller using FreeRTOS. Multiple tasks with different priorities and execution periods are monitored continuously. A watchdog mechanism is incorporated to detect situations in which normal system execution cannot be maintained. Runtime events and diagnostic information are stored in a circular buffer, allowing recent execution history to be retained with limited memory usage. The collected information can also be transmitted through the ST-LINK Virtual COM Port for logging and analysis during development and validation.

An important feature of this project is its ability to perform monitoring and fault localization directly on the target microcontroller. Unlike conventional debugging approaches that depend on external tools or offline analysis, the proposed system performs execution

monitoring and diagnostic evaluation during normal firmware operation. This makes the approach suitable for resource-constrained embedded applications where memory, processing power, and external debugging access are limited.

Overall, this project aims to improve embedded firmware dependability by introducing a continuous, lightweight, and autonomous runtime monitoring approach. By establishing normal execution signatures, detecting deviations, identifying potentially faulty tasks, and providing confidence-based diagnostic information, the proposed system helps reduce the difficulty of locating firmware faults in real-time embedded applications. It also provides a foundation for developing more reliable embedded systems capable of detecting and responding to abnormal execution behaviour with minimal human intervention.

1.1 PROBLEM STATEMENT

To develop a runtime framework for an ARM Cortex-M4 microcontroller running FreeRTOS that detects abnormal firmware execution and automatically identifies the most probable software or hardware component responsible, entirely on the chip and without a PC or external trace hardware during operation. The framework must observe the execution sequence, interrupt timing, task execution time, stack usage, peripheral status, watchdog events, memory-access anomalies and communication errors, and for every detected fault it must report the fault type, the suspected component, a confidence score, the relevant execution history and a recovery action. The research question is whether such lightweight execution signatures can provide useful fault localization without expensive external trace hardware.

1.2 OBJECTIVES

- Capture all eight runtime signals on the microcontroller with low overhead, using only on-chip debug, protection and timing features.
- Learn a baseline of normal behaviour on the board and store it persistently.
- Detect deviations from the baseline with rule-based checks running on the microcontroller.
- Localize each fault to a specific task, interrupt or peripheral and assign a confidence score.
- Recover automatically through a deterministic action and store a fault report for later retrieval.

- Evaluate detection, localization, false alarms, detection latency, CPU overhead and RAM overhead with deliberately injected faults.

1.3 SCOPE OF THE PROJECT

The project covers the complete chain from capture to report on a single STM32F446RE board. The monitored workload is a soft real-time sensor application written for this purpose, and a real-time clock add-on is used to quantify the effect of each fault on that application. Faults are injected deliberately through a command interface, so that every detection can be scored against a known ground truth.

Detection runs autonomously on the microcontroller. A PC is used only to read the UART output, type commands and store logs. Memory-protection faults are limited to stack overflow, writes to a read-only region and NULL-pointer accesses; full control-flow integrity is outside the scope. The statistical comparison is limited to a z-score localizer that runs on the board; offline machine-learning models are discussed only as future work. The evaluation consists of one to two trials per fault class on one board, which demonstrates the mechanisms but does not provide statistically strong detection rates.

CHAPTER 2
LITERATURE REVIEW

Dependability and run-time diagnostics. Avizienis et al. [1] describe how a fault becomes an error and, if nothing intervenes, a failure, and why error detection and recovery are needed for faults that survive development. IEC 61508 [2] and ISO 26262 [3] list run-time diagnostic measures such as program-sequence monitoring, independent watchdogs, RAM and register tests and memory protection. Three of these appear in our framework: the independent watchdog, the stage-sequence check and the register checksums. We do not claim compliance with either standard.

Control-flow monitoring. Mahmood and McCluskey [9] surveyed watchdog processors that compare a program's actual control flow with a reference, and Oh et al. [10] proposed software signatures that are updated in every basic block and checked against expected values. Our stage-transition check uses the same idea at a coarser level: the application marks a few processing stages, and the allowed transitions between them are learned during normal operation instead of being generated by a compiler.

Software fault localization. Wong et al. [7] surveyed 331 papers and grouped the techniques into slice-based, spectrum-based, statistical, program-state, machine-learning and model-based families. Spectrum-based methods such as Tarantula [11] rank statements by how often they execute in failing and passing tests, and Abreu et al. [12] showed that the choice of similarity coefficient strongly affects the result. These methods need many test runs with known outcomes and produce a ranked list for a developer. Our monitor sees one continuously running execution with no test oracle and must choose a single component and a recovery action quickly. Its confidence score plays a role similar to a suspiciousness score, but it is computed from how far a measurement lies outside its learned range.

Fault analysis on ARM firmware. FirmRCA by Chang et al. [8] reconstructs execution events from crashes found by fuzzing; for most of 41 crashes in 17 firmware images it placed the root cause within the top ten instructions. Gabor [13] combined spectrum-based localization with assertions in embedded C/C++ software. Both analyse the failure on a host after it has happened, and FirmRCA targets security bugs rather than faults in a running device. At the hardware level, Yiu [5] and the ARMv7-M documentation [4], [14] describe the Cortex-M fault status registers and stacked exception frames that identify the faulting instruction. Our trap path reads these registers from firmware, without a debugger.

Fault injection. Real faults are too rare to evaluate a monitor with, so faults are injected. Hsueh et al. [15] classify injection techniques as hardware-based, software-based, simulation-based or hybrid. Software-implemented injection is cheap, repeatable and portable, but it reaches only what software can reach; Ziade et al. [16] and Benso and Prinetto [17] review the tools and fault models used for embedded systems. We inject faults in software, inside tasks and from a timer interrupt, and keep a ground-truth record of each injection so that detection and localization can be scored.

Self-test and anomaly detection. CASP by Li et al. [18] tests processor hardware during normal operation using patterns stored in non-volatile memory. It targets hardware defects rather than firmware faults, but it shares the idea of autonomous on-line diagnosis with stored results. Chandola et al. [19] group anomaly detection into statistical, classification, nearest-neighbour and clustering methods; flagging values several standard deviations from the mean is the basis of our statistical localizer. The Isolation Forest of Liu et al. [20] was considered as an offline comparison but was not implemented. Welford's algorithm [21] computes a running mean and variance without storing the samples, which makes learning possible on the microcontroller.

Research gap. Existing work either localizes faults precisely but offline, with a host and many test runs, or detects errors on the device without naming the faulty component. This project combines the two on a small microcontroller: the device detects the fault, names the component with a confidence score, keeps the execution history and recovers.

2.1 COMPARISON OF RELATED WORK

Table 2.1 compares the reviewed approaches with the framework developed in this project.

Table 2.1: Comparison of related work with this project

Work	Target	Where analysis runs	Output	Recovery
Wong et al. survey [7]	General software	Host, offline	Ranked suspicious statements	No
Jones et al., Tarantula [11]	Programs with test suites	Host, offline	Suspiciousness ranking	No
Chang et al., FirmRCA [8]	ARM firmware crashes from fuzzing	Host, offline	Root-cause instructions	No
Gabor [13]	Embedded C/C++ software	Host with injection	Localization with assertions	No
Li et al., CASP [18]	Processor hardware	On chip, on line	Pass/fail of self-test	Partial

Work	Target	Where analysis runs	Output	Recovery
Oh et al. [10]	Control flow	On chip	Control-flow error flag	No

CHAPTER 3
SYSTEM REQUIREMENTS AND SPECIFICATIONS

3.1 FUNCTIONAL REQUIREMENTS

- The system shall record task switches, interrupt entries and exits, application stage marks, peripheral errors and communication errors with cycle-accurate timestamps.
- The system shall build a signature of the system behaviour every 100 ms.
- The system shall learn a baseline from fault-free operation and keep it across resets.
- The system shall detect overruns, missed deadlines, abnormal interrupt rates, stack exhaustion, sequence violations, peripheral configuration changes, peripheral and communication errors, memory-protection violations and watchdog resets.
- For each fault the system shall report the fault type, suspected component, confidence, rules fired, program counter where available and the last 16 events.
- The system shall execute a recovery action from a fixed table and escalate if the same component fails again within 10 s.
- The system shall store each report in flash and list it on request.
- The system shall provide a command interface to inject faults, cancel them, read the ground-truth log, dump the trace and inspect the status.

3.2 NON-FUNCTIONAL REQUIREMENTS

- Detection, localization and recovery must work with no debugger or PC attached.
- Bounded memory: All buffers are statically sized; the monitor must fit beside the application in 128 KB of SRAM.
- Determinism: Recovery actions shall use predefined tables rather than adaptive decisions to ensure predictable system behaviour.
- Persistence: Fault reports and the execution baseline shall remain persistent across resets, watchdog resets, and firmware reflashing.
- Low false-alarm rate: Normal input variations and expected system operation shall not generate false fault alarms.

3.3 HARDWARE SPECIFICATIONS

Table 3.1: Hardware used in the project

Item	Specification	Use in the project
Development board	STM32 NUCLEO-F446RE	Complete target platform
Microcontroller	STM32F446RE, ARM Cortex-M4F, run at 84 MHz from the 16 MHz HSI through the PLL	Application and monitor
Memory	512 KB Flash (8 sectors), 128 KB SRAM	Firmware, baseline (sector 6), reports (sector 7)
Debug and trace	DWT, ITM, FPB, MPU (8 regions), SCB fault registers	DWT, MPU and SCB used autonomously
Watchdog	Independent watchdog clocked by the 32 kHz LSI	Recovery from total hangs
Analog input	ADC1 channel 0 on PA0 (A0), 0–3.3 V	Sensor input of the application
Outputs	LD2 on PA5 (alarm), external LED with 330 Ω on PA6 / D12 (heartbeat)	Application output, liveness indicator
Host link	USART2 at 115200 baud through the ST- LINK virtual COM port	Command shell, reports, telemetry

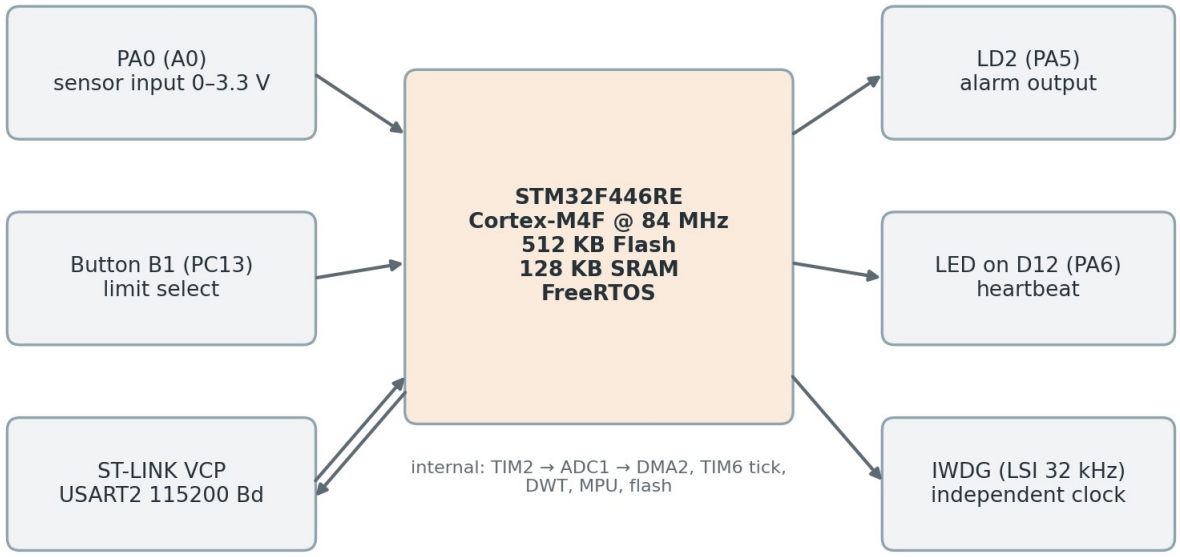


Figure 3.1: Hardware block diagram of the target setup

3.4 SOFTWARE SPECIFICATIONS

Table 3.2: Software tools and libraries

Item	Version / detail	Use
STM32CubeIDE	1.19 with GNU Arm Embedded GCC	Editing, building, flashing
STM32CubeMX (integrated)	HAL drivers, clock and pin configuration	Board initialisation
FreeRTOS	Through the CMSIS-RTOS2 wrapper, preemptive, 1 kHz tick	Task scheduling
CMSIS-Core	Register definitions for DWT, MPU, SCB, NVIC	Low-level access
Serial terminal	PuTTY and Serial Debug Assistant, 115200 8-N-1	Commands and log capture

3.5 SIGNALS TO BE OBSERVED

Table 3.3 shows how each of the eight signals is captured on the board and which rule uses it.

Table 3.3: The eight observed signals and how they are captured

Signal	Captured by	Used by rule
Function / task execution sequence	FreeRTOS switch hooks and application stage marks	Sequence, liveness
ISR timing	Enter and exit wrappers in each monitored ISR, DWT timestamps	ISR rate
Task execution time	DWT cycles between switch-in and switch-out	Overrun
Stack usage	Free-stack watermark and MPU guard regions	Stack, stack overflow
Peripheral status	CRC32 of configuration registers, error flags (ADC overrun, DMA error, timeout)	Peripheral
Watchdog events	Reset-cause register RCC_CSR at boot	Watchdog
Memory-access anomalies	MPU violations reported through MemManage and the CFSR	Trap
Communication errors	USART overrun, framing and noise flags, frame checksum	Communication

CHAPTER 4
SYSTEM ARCHITECTURE AND DESIGN

4.1 DESIGN OVERVIEW

The proposed framework operates using a three-stage approach, as shown in Figure 4.1.

In the first stage, the firmware runs normally while monitoring hooks collect runtime information. This includes task execution details, execution timing, stack usage, peripheral states, error flags, and the cause of the most recent system reset.

In the second stage, the collected information is processed at regular intervals of 100 ms to generate a runtime signature. This signature is compared with a baseline profile stored in flash memory to identify any deviations from normal system behaviour. If no abnormality is detected, the monitoring process continues without interrupting the normal operation of the firmware.

In the third stage, if an abnormality is detected, the framework identifies the potentially faulty component and assigns a confidence score indicating the likelihood of the detected fault. It then generates a diagnostic report containing the recent execution history and performs an appropriate recovery action to restore normal system operation.

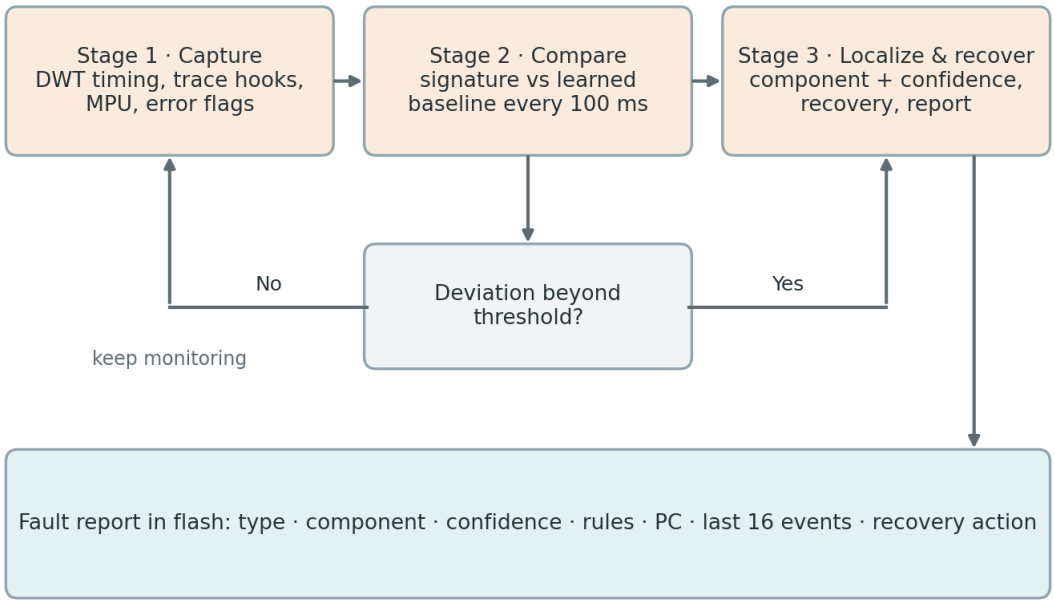


Figure 4.1: Three-stage approach: Capture, Compare, Localize and Recover

4.2 SOFTWARE ARCHITECTURE

The firmware is organised into four main layers, as illustrated in Figure 4.2.

The Application Layer contains the monitored sensor application and the real-time clock (RTC) add-on. It performs the normal application operations and provides the execution environment in which faults are monitored.

The Capture Layer is responsible for collecting runtime information without interfering with the normal execution of the application. Monitoring hooks record events in a ring buffer and update counters during each monitoring window. This information is later used by the detection mechanism.

The Detection Layer operates through a high-priority task. It learns the normal operating behaviour of the system, applies fault-detection rules, identifies potentially faulty components, and initiates the required recovery and reporting procedures.

The Injection and Shell Layer is mainly used for testing and demonstration purposes. It provides facilities to inject faults and interact with the framework through a command-line interface. This layer is not essential for the normal operation of the monitoring mechanism.

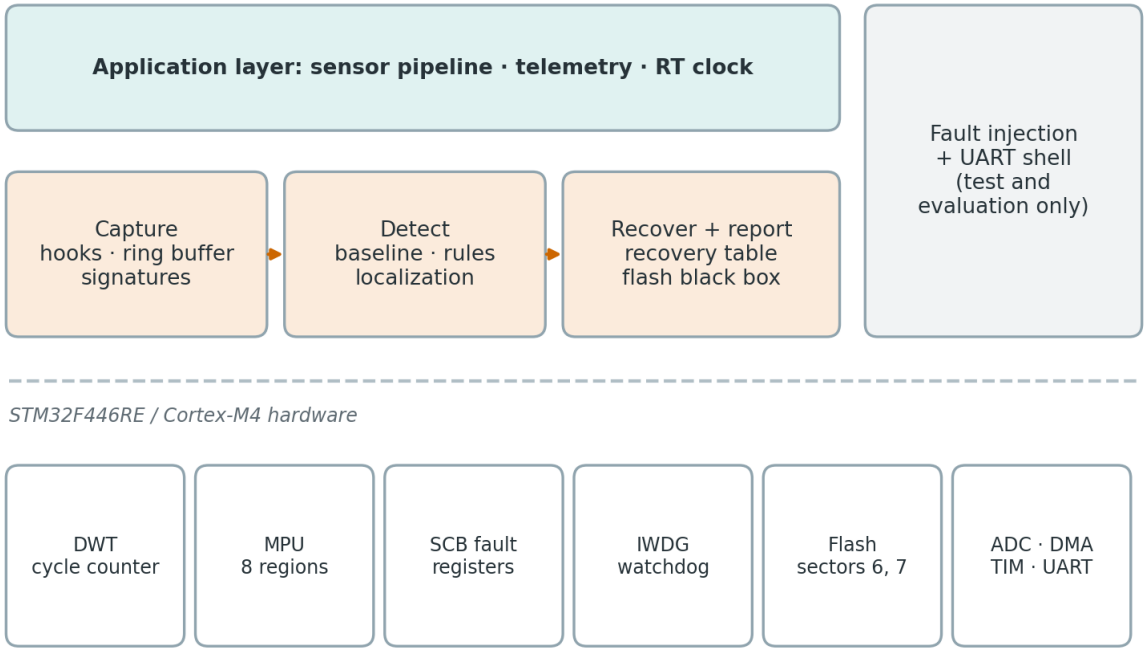


Figure 4.2: Layered software architecture and the hardware features it uses

Below these four layers, the hardware features shown in the lower section provide necessary processor and peripheral support for runtime monitoring, fault detection, and recovery.

The project source code is organised according to the four software layers. The /app directory contains the application-related code, while /monitor contains the runtime monitoring

and data-capture implementation. The /detect directory includes the baseline learning, fault detection, fault localization, recovery, and black box reporting modules. The /inject directory contains the UART driver, command-line shell, and fault injection mechanisms used for testing and demonstration.

The Detection Layer is designed to operate independently of application-specific tasks and peripheral registers. Instead of directly accessing individual application components, the framework uses a component registration mechanism. During system initialization, each task, interrupt, and peripheral registers itself with the monitoring framework using a unique component identifier and name. This registration mechanism allows the framework to identify and monitor different system components without requiring prior knowledge of the application's internal implementation. As a result, the monitoring and fault-localization mechanism can be reused across different embedded applications with minimal modifications.

4.3 TASK AND INTERRUPT MODEL

The system runs on FreeRTOS with preemptive priority scheduling. Table 4.1 lists the tasks and interrupts and the reasoning behind their priorities.

Table 4.1: Tasks and interrupts of the system

ID	Component	Priority	Rate	Role
4	wdg (watchdog feeder)	Realtime	250 ms	Reloads the IWDG only if the monitor has checked in
3	monitor	High	100 ms	Signature, learning, detection, recovery
0	sensor	AboveNormal	10 ms	Application pipeline A → B → C → D
1	report	Normal	100 ms	Stage E: telemetry frame
–	rt_clock	BelowNormal	10 ms	Real-time clock add-on (observer, not monitored)
2	shell	Low	Polled, 100 ms	UART command interface
6	tim6_isr	NVIC 5	1 kHz	Application tick, fires hardware faults
5	adc_isr	NVIC 6	100 Hz	DMA block ready, wakes the sensor task
7	uart_isr	NVIC 7	on event	Receives and sends characters
8	btn_isr	BSP default	on event	User button B1

The watchdog feeder task is assigned the highest priority to prevent it from being starved by other tasks. However, it reloads the hardware watchdog only when the monitoring task has recently reported its successful execution. This ensures that the watchdog is not refreshed if the monitoring mechanism itself becomes unresponsive.

The monitoring task is assigned a higher priority than all application tasks. This allows it to observe the system regularly, even if an application task enters an unexpected execution state or consumes excessive processor time.

The sensor task is given a higher priority than the reporting task because it has more time-critical operations and stricter execution deadlines. The shell and real-time clock (RTC) add-on tasks are assigned lower priorities so that their execution does not interfere with the main application or monitoring activities.

Regarding interrupt configuration, the Nested Vectored Interrupt Controller (NVIC) priority is configured to support FreeRTOS interrupt-safe API functions. With configMAX_SYSCALL_INTERRUPT_PRIORITY set to 5, interrupts with numerical priority values of 5 or greater (lower urgency) are permitted to call FreeRTOS FromISR functions. Interrupts with higher urgency (numerical priority values below 5) must not call these functions.

4.4 SHARED INTERFACES

Three data structures form the frozen interface between the layers. They were fixed before the layers were implemented and are checked at compile time.

The event record, event_t, is exactly 8 bytes: a 32-bit cycle timestamp from the DWT counter, an 8-bit component identifier, an 8-bit event type and a 16-bit extra field whose meaning depends on the type (stage number, interrupt duration in microseconds, fault status bits). Keeping the record small allows 1024 events in 8 KB.

The fault report, fault_report_t, is a fixed 164-byte record: a sequence number, the fault type, the suspected component, the confidence, the localizer that decided, the rules that fired, the recovery action, the program counter and fault status registers where available, the reset cause, timestamps and the last 16 events. A static assertion fixes its size because it is stored in flash and read back by later firmware versions.

The component identifiers form one shared enumeration: 0 sensor, 1 report, 2 shell, 3 monitor, 4 watchdog feeder, 5 ADC interrupt, 6 TIM6 interrupt, 7 UART interrupt, 8 button interrupt, 9 ADC1, 10 TIM2, 11 GPIOA and 12 USART2. Up to 16 components are supported.

4.5 MEMORY LAYOUT

Figure 4.3 shows how flash and SRAM are used. The linker script stops the firmware at the end of sector 5, so the baseline in sector 6 and the fault reports in sector 7 cannot be

overwritten when the program grows, and they survive reflashing because the programmer only erases the sectors it writes. In SRAM, a .noinit section, which the start-up code does not clear, holds the crash record, the last execution history, the boot counter and the last injection. The MPU protects address zero against NULL-pointer accesses, makes a 32-byte configuration block read-only and places a 32-byte guard below the stack of each monitored task.

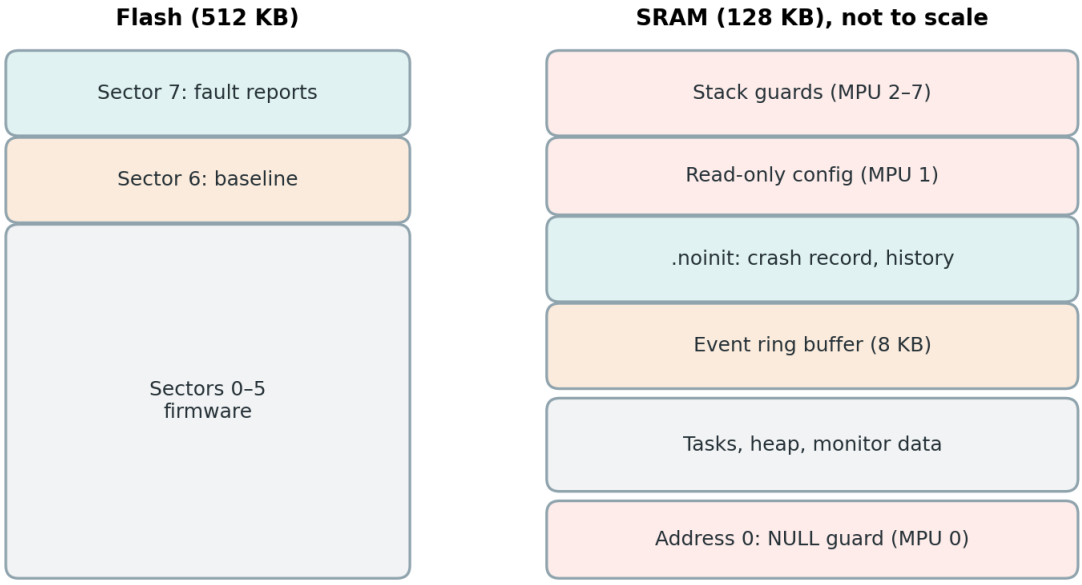


Figure 4.3: Use of flash and SRAM, including the MPU-protected regions

4.6 DESIGN DECISIONS

A monitoring window of 100 ms was selected to balance detection speed and processor usage. A shorter window can detect faults faster but requires more processing time and may collect fewer events for statistical analysis. A longer window reduces processing overhead but increases fault-detection time. Since the application has a task period of 10 ms, a 100 ms window contains ten complete sensor task cycles. This provides sufficient data for monitoring while keeping the monitor's CPU usage close to 1%.

The framework uses separate mechanisms for fault detection and event storage. Per-window counters are used for detection because they retain the collected counts throughout each monitoring window. A ring buffer stores recent events for diagnostic purposes and may overwrite older entries when it becomes full. Therefore, even if the ring buffer is full, fault detection continues without losing the counter information.

The baseline is learned directly on the STM32 board instead of being generated on a computer and programmed into the firmware. This ensures that the learned baseline represents

the actual operating conditions of the device, including its firmware version, clock configuration, and interrupt activity. It also simplifies the development and testing process.

A fixed recovery mechanism was selected instead of an adaptive recovery system. Adaptive recovery may change its actions based on previous experiences, making its behaviour more difficult to predict and verify. In contrast, the proposed framework uses predefined recovery actions with an escalation mechanism. This makes the system behaviour predictable and easier to test under different fault conditions.

The monitoring task is assigned a higher priority than all application tasks so that it can continue monitoring even when an application task behaves unexpectedly. The watchdog feeder is assigned the highest priority and refreshes the hardware watchdog only after receiving confirmation that the monitoring task is functioning correctly. This arrangement ensures that a runaway application task cannot prevent fault detection and that a failure in the monitoring task can eventually lead to a system reset.

CHAPTER 5
THE MONITORED APPLICATION

5.1 PURPOSE OF THE APPLICATION

A practical application is needed to test the proposed fault-monitoring framework. For this purpose, a soft real-time sensor-monitoring application is used.

The application reads an input voltage, filters the sampled data, compares it with a predefined alarm limit, controls an alarm output, and sends telemetry data. Although the application is simple, it uses important embedded system features such as hardware timers, ADC, DMA, interrupts, multiple tasks with different priorities, and a communication interface. This makes it suitable for testing the monitoring framework under both normal and faulty operating conditions.

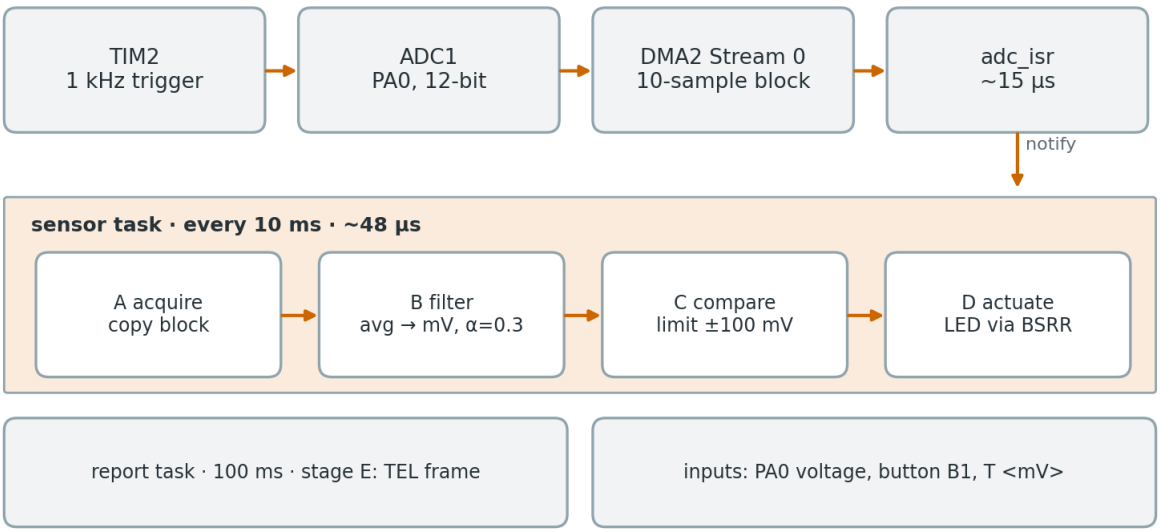


Figure 5.1: Data path of the sensor application from the timer trigger to the alarm LED

5.2 HARDWARE-TIMED ACQUISITION

TIM2 is configured to produce an update event at 1 kHz, and this event triggers one ADC1 conversion of channel 0 (PA0) in hardware, so the sampling rate does not depend on software timing. DMA2 Stream 0 moves each 12-bit result into a buffer of ten samples. When ten samples have been collected, every 10 ms, the DMA transfer-complete interrupt fires. The interrupt handler checks the DMA transfer-error and ADC overrun flags, copies the block, increments a block sequence counter and wakes the sensor task with a FreeRTOS task notification. Its measured execution time is about 15 µs.

5.3 THE SENSOR TASK

The sensor task waits for the notification with a 50 ms timeout. If no block arrives within that time, the sampling chain is broken and a timeout error is reported against ADC1. Otherwise the task runs four stages, each marked for the monitor:

- Stage A, Acquire: take the latest block of ten samples.
- Stage B, Filter: average the block, convert it to millivolts (× 3300 / 4095) and apply exponential smoothing with α = 0.3.
- Stage C, Compare: switch the alarm on above the limit (default 2000 mV) and off only below the limit minus 100 mV, so that the output does not flicker near the limit.
- Stage D, Actuate: write the alarm state to LED LD2 with a single write to the GPIO bit set/reset register.
-

5.4 TELEMETRY AND INPUTS

The report task runs every 100 ms and is marked as Stage E. It generates telemetry frames containing the frame number, timestamp, alarm limit, input voltage, and alarm status. An XOR checksum is added to each frame to detect communication errors. Only every tenth frame is printed, resulting in one telemetry message per second. The alarm limit can be changed using the user button, which cycles through 1000, 1650, 2000, and 2500 mV. It can also be changed using the T command through the UART interface.

5.5 COMMAND INTERFACE

The system uses single-letter commands through USART2 for user interaction. Each module registers its commands during system startup, allowing the shell to handle commands without depending on individual modules. Commands can also be sent in a framed format using $command*hh, where hh represents the XOR checksum. If the checksum does not match, the command is rejected and a communication error is reported.

The commands supported by the project are listed in Table 5.1.

Table 5.1: Command interface

Command	Function
H	List all commands
S	Live status: per-component signature, peripheral checksums, ring-buffer state, overhead line
V 1 / V 0	Stream one signature line per window on or off
D	Dump the event ring buffer as CSV
L n	Learn the baseline from n windows (default 100)
B	Print the learned baseline
R / R n	List stored fault reports / show one report with its history
C	Clear the fault log in flash
M 0 / M 1	Acting localizer: rule-based or statistical
K k	Detector sensitivity k (default 4)
I f c d	Arm fault f on component c after d ms
X	Cancel a pending fault
G	Print the ground-truth injection log
W a v	Write value v to address a (examiner tool)
T mV, Y 1/0	Alarm limit; telemetry on/off
Z, Z R, Z 0/1	Clock status; clock resync; clock line off/on

CHAPTER 6
ON-CHIP CAPTURE USING CORTEX-M4 FEATURES

6.1 ARMV7-M DEBUG AND FAULT ARCHITECTURE

The Cortex-M4 processor includes a built-in debug subsystem [4], [14]. Some of its features require an external debugging tool. For example, the Instrumentation Trace Macrocell (ITM) and Serial Wire Output (SWO) send debugging information through an output pin, while the Embedded Trace Macrocell (ETM) provides instruction-level execution trace. The Flash Patch and Breakpoint (FPB) unit is mainly used to set breakpoints during debugging.

In this project, only the features that can operate directly through firmware without an external debugger are used. The selected features are summarised in Table 6.1.

Table 6.1: Debug and protection features of the Cortex-M4 and how they are used

Feature	Normal purpose	Used here	Purpose in this project
DWT cycle counter	Profiling with a debugger	Yes	Timestamp of every event
CoreDebug DEMCR	Enable debug blocks	Yes	TRCENA bit powers the DWT
MPU	Memory isolation	Yes	NULL guard, read-only block, stack guards
SCB fault registers	Fault diagnosis	Yes	CFSR, HFSR, MMFAR, BFAR in crash records
DBGMCU (STM32)	Freeze peripherals while halted	Yes	IWDG stops when a debugger halts the core
ITM / SWO	printf-style trace to a host	No	Needs a probe and host
ETM	Instruction trace	No	Needs a trace port
FPB	Breakpoints	No	Debugger only

6.2 TIMESTAMPS FROM THE DWT CYCLE COUNTER

The Data Watchpoint and Trace (DWT) unit contains a 32-bit cycle counter called CYCCNT, which continuously counts CPU clock cycles. It is enabled by setting the TRCENA bit in the CoreDebug DEMCR register, followed by the CYCCNTENA bit in the DWT_CTRL register.

The counter can be read using a single instruction, making it suitable for recording timestamps during every task context switch. At a CPU frequency of 84 MHz, each count represents approximately 11.9 ns. Since it is a 32-bit counter, it wraps around after approximately 51.1 seconds.

To calculate time intervals, the framework uses unsigned subtraction. This method provides correct interval measurements even when the counter wraps around once between two readings.

Table 6.2: DWT registers used

Register	Address	Bit used
DEMCR	0xE000EDFC	24, TRCENA
DWT_CTRL	0xE0001000	0, CYCCNTENA
DWT_CYCCNT	0xE0001004	32-bit count

6.3 TRACE HOOKS

FreeRTOS provides two trace macros, traceTASK_SWITCHED_IN and traceTASK_SWITCHED_OUT, which are called during task context switching. In this project, these macros are connected to the framework's monitoring hook functions through FreeRTOSConfig.h [22].

Each monitored task is assigned a component identifier using the uxTaskNumber field in its Task Control Block (TCB). The identifier is stored as ID + 1, so that zero represents an unmonitored task. Interrupt handlers are monitored using FM_ISR_ENTER and FM_ISR_EXIT, while application stages are recorded using FM_STAGE.

Listing 6.1: Redirecting the kernel trace points

```
// FreeRTOSConfig.h: call our hooks on every context switch
#define traceTASK_SWITCHED_IN()  \
    fm_hook_task_in(pxCurrentTCB->uxTaskNumber)
#define traceTASK_SWITCHED_OUT() \
    fm_hook_task_out(pxCurrentTCB->uxTaskNumber)
// uxTaskNumber holds the component ID + 1 (0 = not monitored)
```

Each monitoring hook performs two operations while briefly disabling interrupts. First, it records an event in the ring buffer. Second, it updates the current monitoring window counters for the corresponding component. These counters include the number of executions, total CPU cycles used, longest execution time, and longest interval between two executions.

High-frequency interrupts, such as the 1 kHz TIM6 interrupt and UART interrupt, are included in the monitoring statistics but excluded from the event history. This prevents frequent events from overwriting more useful diagnostic information in the ring buffer.

6.4 EVENT RING BUFFER

The framework uses a ring buffer to store recent runtime events. It can hold 1024 events, with each event occupying 8 bytes, giving a total storage size of 8 KB. The buffer is placed in the .noinit memory section so that recent event data remains available even after a watchdog reset. Since the buffer size is a power of two, a bitwise AND operation is used to update the buffer index instead of division. This reduces processing time during frequent operations such as task context switching. When the buffer becomes full, the oldest event is overwritten, and an overwrite counter is updated to track the number of lost events.

The push function saves the current PRIMASK value before disabling interrupts and restores the original value after completing the operation. This ensures that interrupts are not enabled unintentionally when the function is called during a FreeRTOS context switch.

Listing 6.2: Pushing an event into the ring buffer

```
void rb_push(ring_buffer_t *rb, const event_t *e)
{
    uint32_t pm = __get_PRIMASK();   // remember the interrupt state
    __disable_irq();                  // write without being interrupted
    rb->buf[rb->head] = *e;           // store the 8-byte event
    rb->head = (rb->head + 1) & 1023; // wrap around after 1024 entries
    if (rb->head == rb->tail) {       // buffer full:
        rb->tail = (rb->tail + 1) & 1023;  // drop the oldest event
        rb->dropped++;                     // and count it
    }
    __set_PRIMASK(pm);                // restore the interrupt state
}
```

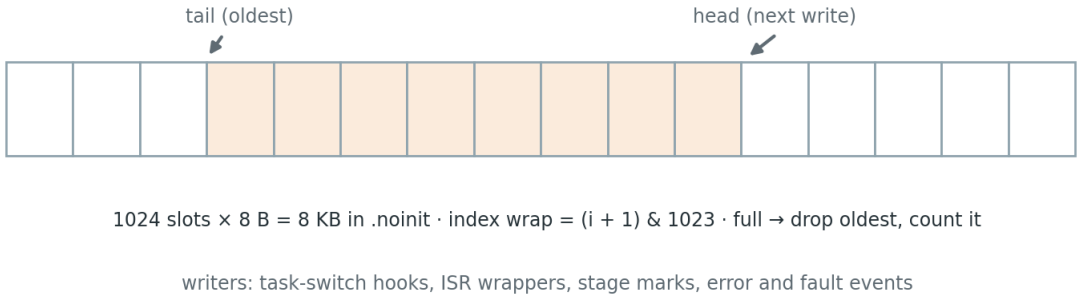


Figure 6.1: Organisation of the event ring buffer

The ring buffer has four main uses:

D command: Dumps the stored events in CSV format for analysis.

Fault report: Copies the latest 16 events into the fault report.

Fault handler: Saves the latest 16 events in .noinit memory before performing a reset.

Monitor: Saves an event snapshot into .noinit at the end of every monitoring window.

This helps preserve recent execution history during a watchdog reset, even when no fault handler gets a chance to execute.

The fault-detection mechanism does not directly use the ring buffer. Instead, it uses monitoring window counters because they require less processing time and do not lose information when the ring buffer overwrites old events.

6.5 THE 100 MS SIGNATURE WINDOW

Every 100 ms, the monitoring task collects the accumulated counters and combines them into a signature vector. After creating the signature, the counters are reset for the next monitoring window. For each component, the signature contains the number of executions, total CPU cycles, longest execution time, and longest interval between executions. If a component does not execute during a monitoring window, the time until the end of that window is considered its execution gap. This helps detect tasks that have stopped running unexpectedly.

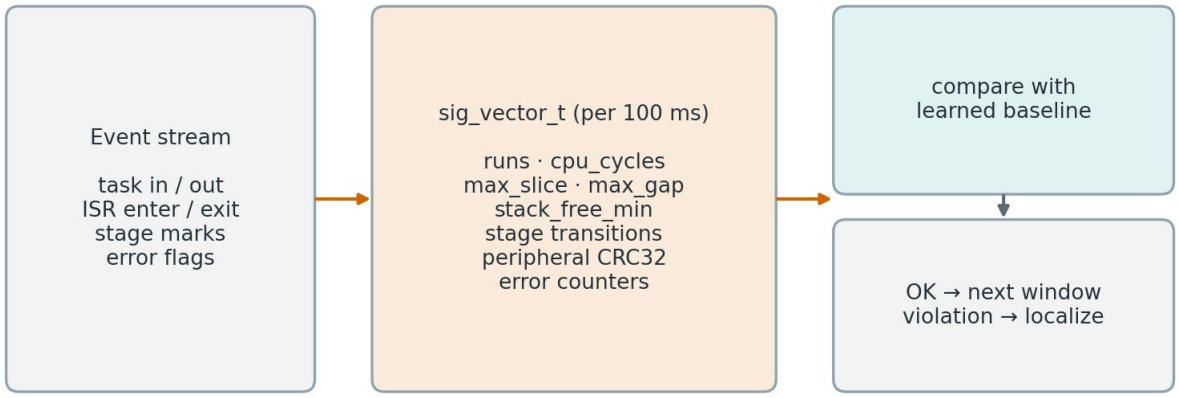


Figure 6.2: Construction of the signature vector in each 100 ms window

The signature vector also includes the remaining stack space of each task, stage-transition counts, CRC32 checksums of peripheral configuration registers, and error counters. Only the important bits of the peripheral registers are included in the checksum calculation. During normal operation, the sensor task executes 10 times within a 100 ms window, and the ADC interrupt also occurs 10 times. The TIM6 interrupt executes approximately 100 times, while the

report task executes once. The stage transitions A → B, B → C, C → D, and D → A each occur 10 times.

These regular execution patterns represent the normal behaviour of the application and are used to create a baseline signature for detecting abnormal conditions.

6.6 MEMORY PROTECTION UNIT

The MPU checks every load and store in hardware at no run-time cost. Its regions must be a power of two in size and aligned to their size, which is why the protected configuration block is exactly 32 bytes and 32-byte aligned. The PRIVDEFENA bit keeps the default memory map for all addresses not covered by a region, so only the protected regions change behaviour.

Table 6.3: MPU regions used

Region	Covers	Size	Access	Catches
0	Address 0x0000_0000	1 KB	None	NULL-pointer access
1	Application configuration block	32 B	Read-only	Stray writes to settings
2–7	Bottom of each monitored task stack	32 B	None	Stack overflow

The stack guards make stack-overflow detection immediate. A stack grows downwards, and without protection an overflowing task silently overwrites whatever lies below its stack. With a no-access guard there, the first write into the guard raises a MemManage fault before any neighbouring memory is touched. FreeRTOS's own stack check (configCHECK_FOR_STACK_OVERFLOW = 2) is kept as a second line of defence.

6.7 FAULT EXCEPTIONS AND THE TRAP PATH

The MemManage, BusFault and UsageFault exceptions are enabled separately in the SHCSR register, and divide-by-zero is made to trap through CCR.DIV_0_TRP. On a fault the core pushes eight registers onto the active stack (Figure 6.3) and loads a special EXC_RETURN value into LR. Bit 2 of EXC_RETURN tells whether the process stack was in use, which means that a task was running. The stacked PC at offset 24 is the exact faulting instruction. The configurable fault status register CFSR tells what went wrong: its low byte is the MemManage status, with DACCVIOL (bit 1) for a data access violation and MMARVALID (bit 7) when MMFAR holds the faulting address [5].

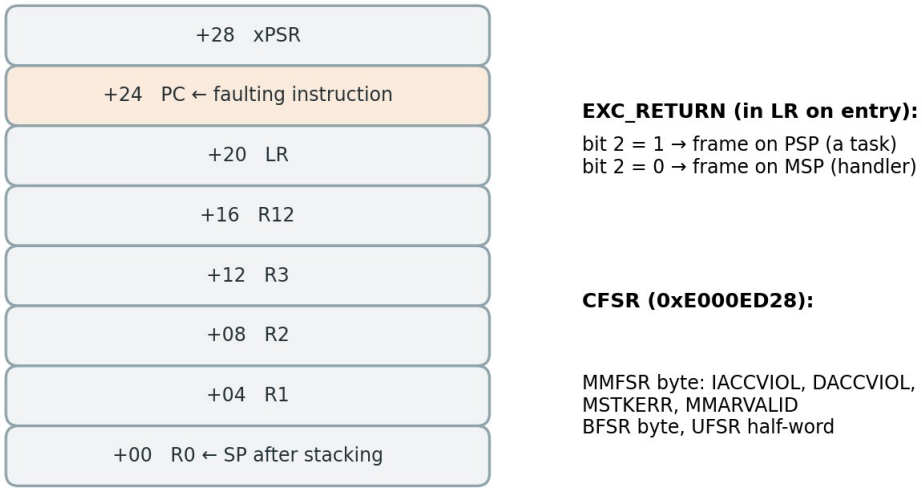


Figure 6.3: Hardware-stacked exception frame and the registers used to diagnose a fault

The handlers themselves are short: they read LR and call fm_trap. This function reads CFSR and the fault address, selects the correct stack from EXC_RETURN, takes the stacked PC unless the fault happened during stacking, classifies the fault as a stack overflow if the address lies inside a stack guard and as a memory-access fault otherwise, copies the last 16 events into .noinit and resets the chip. After the reboot, the monitor turns this record into a normal fault report.

Listing 6.3: Fault handler and trap path (condensed)

```
void MemManage_Handler(void)
{
    uint32_t lr;
    __asm volatile ("mov %0, lr" : "=r"(lr)); // EXC_RETURN
    fm_trap(FM_TRAP_MEMMANAGE, lr);
}

// simplified: the MMARVALID and MSP/PSP checks are left out
void fm_trap(uint8_t kind, uint32_t exc_return)
{
    uint32_t addr = SCB->MMFAR;             // accessed address
    uint32_t *frame = (uint32_t *)__get_PSP();  // task stack
    uint32_t pc = frame[6];                 // faulting instruction
    save_crash_record(kind, addr, pc);      // kept in .noinit RAM
    NVIC_SystemReset();                     // report after reboot
}
```

6.8 RESET CAUSE AND RESET-SURVIVING RAM

After every reset, the firmware first reads the RCC_CSR register to identify the reset cause. It checks whether the reset occurred due to power-on, an external reset pin, a software reset, or the independent watchdog [6]. The flags are then cleared.

During a power-on reset, the .noinit memory area is cleared. For other types of reset, its contents are checked and preserved. This memory area stores the crash record, the latest 16 events, boot counter, last running component, and most recent fault-injection record. Before starting the scheduler, the firmware also saves the identifier of the last running component. This information is later used by the watchdog to generate a fault report.

6.9 PERIPHERAL AND COMMUNICATION ERRORS

Each registered peripheral provides a list of configuration registers and their corresponding bit masks. These include ADC1 trigger settings, TIM2 prescaler, PA0 and PA5 GPIO configurations, and USART2 sampling settings. The monitoring framework calculates a CRC32 checksum of these selected register values during every monitoring window and compares it with the baseline value. Any mismatch indicates a possible change in peripheral configuration.

The framework also monitors different error conditions. The ADC interrupt reports DMA transfer errors and ADC overruns, while the sensor task reports sampling timeouts. The UART driver detects overrun, framing, and noise errors, along with telemetry frame checksum mismatches.

CHAPTER 7
BASELINE LEARNING AND DETECTION

7.1 THE MONITOR TASK

The monitor is a high-priority FreeRTOS task that executes every 100 ms using osDelayUntil. This maintains a fixed monitoring interval without timing drift. During each cycle, the monitor collects the signature data, records the available stack space, checks in with the watchdog feeder, and processes pending shell commands such as baseline learning and log clearing.

As shown in Figure 7.1, the monitor initially ignores the first ten windows because the system may not be stable during startup. If a valid baseline is available in flash memory, it directly enters the monitoring mode. Otherwise, it starts the baseline learning process. After performing a recovery action, the monitor ignores the next three windows to allow the system to return to normal operation.

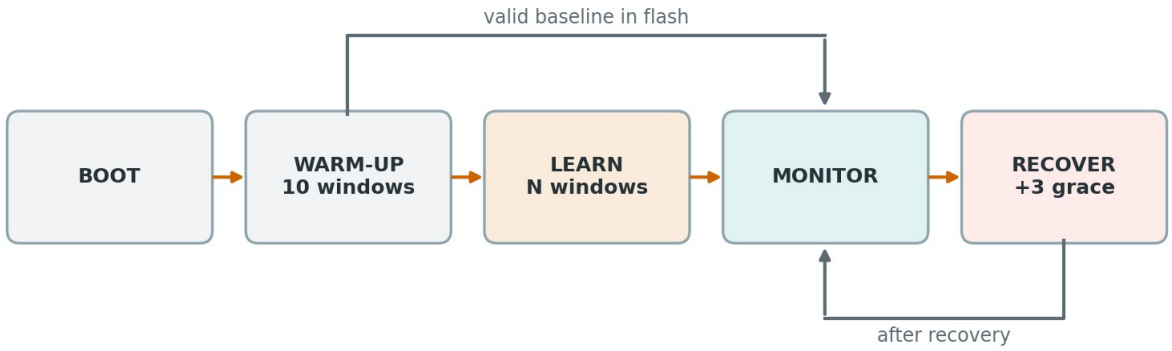


Figure 7.1: Life cycle of the monitor task

7.2 BASELINE LEARNING

Storing every signature would need too much SRAM, so the framework uses Welford's online algorithm [21]. It calculates the running mean and standard deviation without storing all previous samples, making it suitable for resource-constrained embedded systems.

For each component, the baseline stores the mean, standard deviation, and maximum values of the execution count, CPU cycles, and longest execution time. It also records the longest execution gap, minimum available stack space, and whether the component follows a periodic execution pattern. The baseline also stores the observed stage transitions, peripheral configuration CRC values, and a fingerprint of the registered components. Once learning is

completed, the baseline is saved in flash memory sector 6 along with a CRC32 checksum. If the registered component set changes, the existing baseline is automatically rejected.

Listing 7.1: Welford update used during learning

```
// called once per window for each learned value x
void fm_welford_add(fm_welford_t *w, float x)
{
    w->n++;
    float d = x - w->mean;
    w->mean += d / w->n;           // new running mean
    w->m2   += d * (x - w->mean);  // new sum of squared deviations
}
// standard deviation = sqrt(m2 / (n - 1))
```

Learning runs for 100 windows (10 s) by default. Faults must not be injected while learning, or the faulty behaviour becomes part of "normal".

7.3 THRESHOLDS

The framework uses different thresholds to identify abnormal behaviour. Each threshold is calculated using the mean, standard deviation, maximum observed value, and a minimum fixed limit. The default sensitivity value is set to 4 and can be changed using the K command. During detection, the measured value is divided by its threshold to calculate a margin. If the margin is greater than 1, the corresponding component is considered abnormal.

Table 7.1: Detection thresholds

Feature	Threshold (largest term applies)	Applies to
CPU time per window	max(μ + kσ, 1.5 × max, 5 % of the window)	Periodic tasks
Longest single run	max(μ + kσ, 2 × max, 1 ms)	Periodic tasks
Event-driven task	max(50 % of the window, 1.5 × max)	Irregular tasks
Interrupt rate	max(μ + kσ, 1.5 × max) + 2	Periodic interrupts
Liveness	2 × longest gap seen + 2 ms	Periodic, regular components
Stack	free < 16 words and below baseline	Tasks

For example, the sensor task normally executes every 10 ms. Its liveness threshold is calculated as 22 ms (2 × 10 + 2). If the task remains inactive for 100 ms, the resulting margin is approximately 4.5, indicating a significant deviation from normal execution.

7.4 DETECTION RULES

The framework uses two types of detection rules: timing-based rules and hard-evidence rules, as shown in Table 7.2. Timing rules compare measurements with the learned thresholds;

hard-evidence rules react to direct indications such as an unseen stage transition, a changed register or a processor fault.

Hard evidence is given higher priority than timing-based evidence because timing abnormalities may occur as a result of another underlying fault. For example, if the ADC stops working, the sensor task may remain inactive while waiting for new samples. In this case, the actual fault is associated with the ADC rather than the sensor task.

Table 7.2: Detection rules and their classes

Bit	Rule	Class	Typical fault
0x0001	OVERRUN	Timing	Task running far too long
0x0002	LIVENESS	Timing	Task or ISR silent too long
0x0004	ISR_RATE	Timing	Interrupt storm
0x0008	STACK	Hard	Stack nearly exhausted
0x0010	SEQUENCE	Hard	Stage transition never seen in learning
0x0020	PERIPH_CFG	Hard	Configuration register changed
0x0040	PERIPH_ERR	Hard	ADC overrun, DMA error, sampling timeout
0x0080	COMM	Hard	UART error flag or frame checksum
0x0100	TRAP	Hard, after reset	MemManage, BusFault, UsageFault
0x0200	WATCHDOG	Hard, after reset	Independent watchdog reset
0x0400	STAT	Statistical	Sum of squared z-scores above threshold

7.5 STATISTICAL LOCALIZER

The statistical localizer does not use the rule table. For every task and interrupt it computes a z-score for four features: run count, CPU time, longest run and longest gap.

A minimum standard deviation is used in the calculation to prevent very small variations from producing extremely large z-scores. The squared z-scores of all four features are then added to calculate the total anomaly score (S), as shown in Equations (7.1) and (7.2).

The score S is compared with a threshold T, which is 128 for the default sensitivity k = 4. A component with S > T becomes a candidate, and the candidate with the highest score is selected as the most likely faulty component.

$$z_f = (x_f - \mu_f) / \max(\sigma_f, 0.05 \cdot \mu_f, 1) \quad (7.1)$$

$$S = \Sigma\, z_f^2 \quad \text{(over runs, CPU time, longest run, gap)} \quad (7.2)$$

$$T = 2 \cdot (2k)^2 = 128 \quad \text{for } k = 4 \quad (7.3)$$

Both localizers run in every monitoring window. The M command selects which one makes the final decision; the other prints its verdict as a !SHADOW message, so the two methods can be compared on the same data.

CHAPTER 8
LOCALIZATION, CONFIDENCE AND RECOVERY

8.1 LOCALIZATION PIPELINE

Each component that violates a detection rule is identified as a candidate with a fault type and a corresponding margin. These candidates pass through four filtering stages, as shown in Figure 8.1. First, hard evidence is given priority, and timing-based candidates that are only symptoms of another fault are removed. Next, the debounce mechanism checks whether a timing violation continues for two consecutive monitoring windows, unless its margin is 3 or higher. The victim suppression mechanism removes lower-priority tasks that become inactive because a higher-priority task is consuming the CPU. Finally, the candidate with the highest margin is selected as the most likely faulty component.

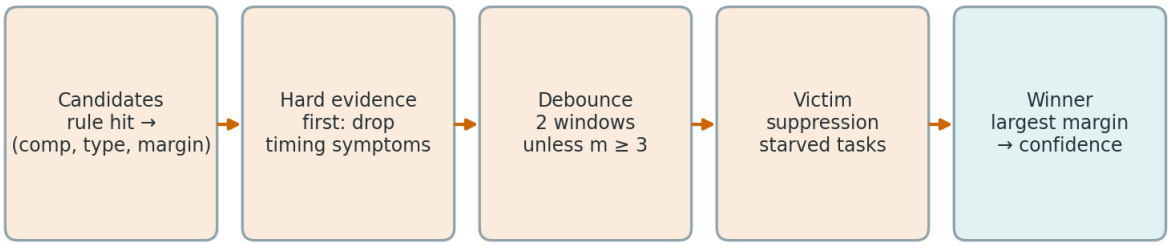


Figure 8.1: Localization pipeline from rule hits to the winning candidate

8.2 CONFIDENCE SCORING

The confidence score indicates the strength of the evidence supporting a detected fault. In the rule-based localizer, hard evidence is assigned a fixed confidence value, as shown in Table 8.1. For timing-based faults, the confidence is calculated using the logarithm of the margin (m), as given in Equation (8.1).

$$conf = 55 + 15 \cdot log_2(m) \quad (8.1)$$

A margin twice the threshold increases the confidence by 15 points, while a margin four times the threshold increases it by 30 points. If multiple candidates remain, 10 points are deducted for each additional candidate. The final confidence value is limited to a range of 30% to 99%.

For the statistical localizer, confidence is calculated using the anomaly score (S) and threshold (T), as shown in Equation (8.2).

$$conf = 100 \cdot S / (S + T) \quad (8.2)$$

A score equal to the threshold gives 50% confidence, while scores three and nine times the threshold give 75% and 90% confidence, respectively.

Table 8.1: Confidence assigned to each type of evidence

Evidence	Confidence
Stack-guard hit after reset	99 %
Configuration CRC changed, peripheral error flag, memory trap	95 %
Stage transition never seen, UART checksum error	90 %
Watchdog reset with the last task known / unknown	80 % / 40 %
Timing evidence (rule-based)	$55 + 15 \cdot \log_2(m)$
Statistical localizer	$100 \cdot S / (S + T)$
Each additional surviving candidate	−10 %

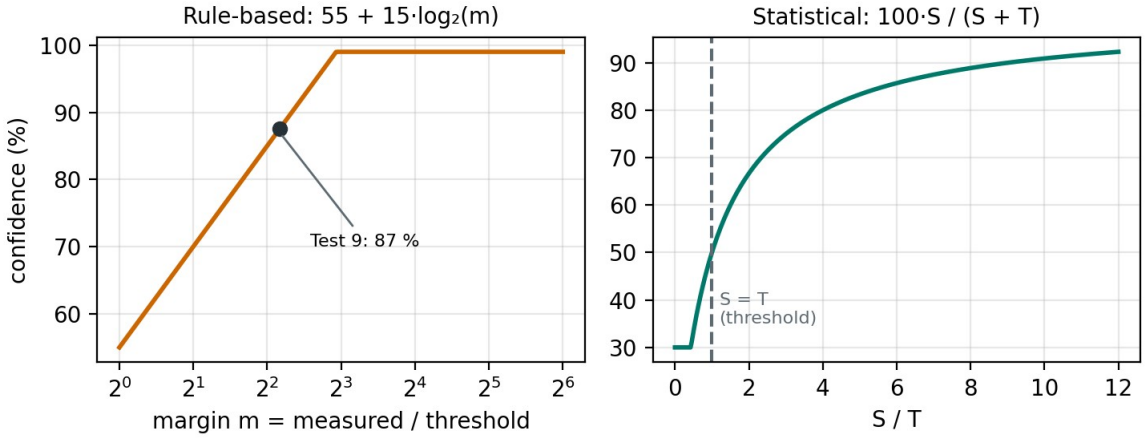


Figure 8.2: Confidence as a function of the margin (rule-based) and of S/T (statistical)

As a worked example, the missed-deadline test of Chapter 12 left the sensor task silent for 100 ms against a liveness limit of 22 ms, a margin of about 4.5. The formula gives $55 + 15 \times \log_2(4.5) \approx 87$ %, which is exactly the confidence reported by the board.

8.3 RECOVERY ENGINE

Recovery is table-driven and deterministic (Table 8.2). Task faults restart the task through its registered restart function, which deletes and recreates it. Peripheral and interrupt faults pulse the peripheral's reset line in RCC and reconfigure it exactly as at boot. Memory traps and watchdog resets are logged only, because the reset has already happened. If the same component fails again within 10 s, the simple fix evidently did not hold, so the engine escalates (Figure 8.3).

Table 8.2: Recovery table

Fault class	Action
Task (overrun, liveness, sequence, stack)	RESTART_TASK

Fault class	Action
Peripheral, interrupt, communication	REINIT_PERIPH: RCC reset and reconfiguration
Trap, watchdog	Reported after the reset
Same component again within 10 s	Level 1: also re-initialise all peripherals; level 2: system reset

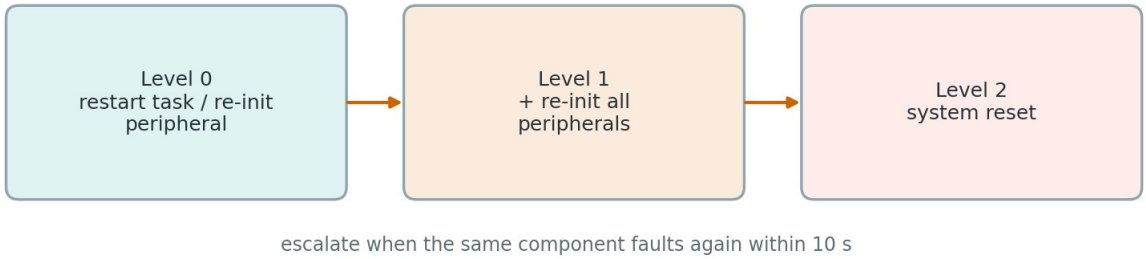


Figure 8.3: Escalation of recovery for repeated faults on the same component

8.4 WATCHDOG DESIGN

The independent watchdog uses a 32 kHz Low-Speed Internal (LSI) oscillator, which operates independently of the main system clock. With a prescaler of 64 and a reload value of 2000, the watchdog resets the microcontroller after approximately 4 seconds if it is not refreshed.

The watchdog feeder task runs at the highest priority and refreshes the watchdog every 250 ms, but only if the monitor task has checked in within the last 1.5 seconds. If the monitor stops responding, the watchdog triggers a system reset. This protects the system against both application failures and monitor failures. During debugging, the watchdog is frozen when the processor is halted. Therefore, watchdog testing must be performed without an external debugger connected.

8.5 FAULT REPORT AND FLASH BLACK BOX

Whenever a fault is detected, the framework generates a 164-byte fault report. The report is transmitted through UART as a !FAULT message, followed by !HIST messages containing recent execution history.

The reports are also stored in flash memory sector 7, which can hold up to 512 reports. The sector is erased only when it becomes full, allowing previous fault records to remain available even after power loss or firmware reflashing.

If a fault causes the processor to stop, the trap handler or watchdog first stores the available fault information in .noinit memory. During the next boot, the monitor retrieves this information and generates the corresponding fault report in flash memory.

Table 8.3: Contents of a fault report

Field	Meaning
seq, type, comp	What happened and where
conf, loc	How sure, and which localizer decided
action, rules	What was done and which rules fired
pc, cfsr, addr	Faulting instruction and fault registers (traps)
reset cause, uptime	Context
16 × event_t	Execution history before the fault, with relative timestamps

CHAPTER 9
FAULT INJECTION FRAMEWORK

9.1 PROPOSED ARCHITECTURE

Real firmware faults are difficult to reproduce and evaluate because they occur rarely and unpredictably. Therefore, deliberate fault injection is used to test the monitoring framework.

The fault injector is implemented in software and controlled through the command shell using the command I fault component delay. Before injecting a fault, the requested command is checked against a predefined table of valid fault types and target components. The validated request is then stored in a small record while interrupts are briefly disabled.

Task-related faults are triggered inside the target task when it reaches its injection point. These faults execute using the target task's own stack. Hardware-related faults are triggered through the 1 kHz TIM6 interrupt. Whenever a fault occurs, an INJECT event is recorded in the trace, and the details are stored in RAM and .noinit memory.

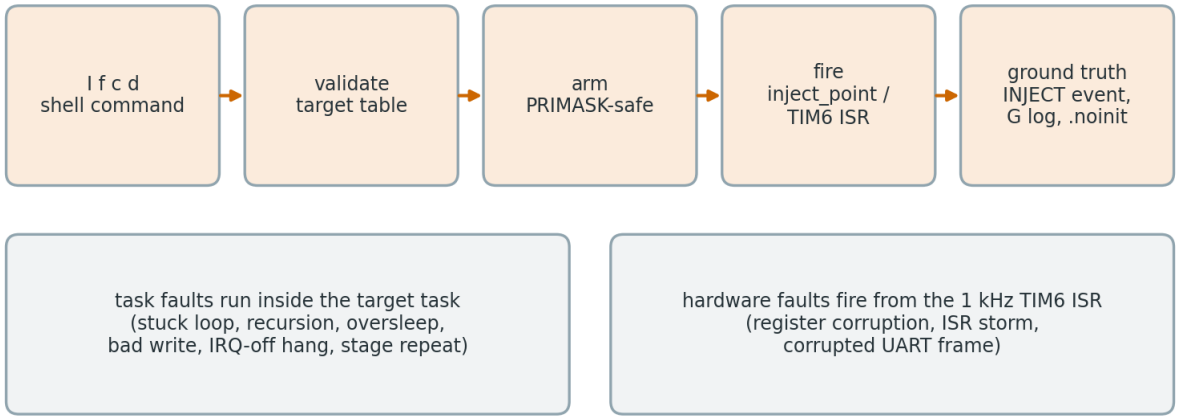


Figure 9.1: Path of a fault from the command to the ground-truth record

The monitoring framework does not receive information about the injected fault. It identifies the fault only by observing its effects on system behaviour. After localization, the actual injected fault is compared with the identified component using a !GT message. The G command displays the ground-truth log, which records when a fault is scheduled and when it occurs. The X command cancels a fault that has not yet been triggered.

9.2 FAULT CATALOGUE

The project uses ten different fault types to test the eight monitored signals, as listed in Table 9.1. Each fault type is restricted to specific components where that fault can occur meaningfully. The I command displays the valid component identifiers for each fault type. If an invalid target is selected, the command is rejected, and the shell displays the list of valid targets.

Table 9.1: Injectable faults

ID	Fault	Mechanism	Valid targets	Expected detection
1	STUCK_LOOP	Infinite loop with interrupts enabled	0, 1, 2	EXEC_OVERRUN
2	STACK_OVERFLOW	Unbounded recursion into the MPU guard	0, 1, 2	STACK_OVERFLOW
3	MISSED_DEADLINE	Task sleeps 10 × its period	0, 1	LIVENESS
4	ISR_STORM	TIM6 from 1 kHz to 10 kHz with extra work	6	ISR_RATE
5	MEM_RO_WRITE	Write to the MPU read-only block	0, 1, 2	MEM_ACCESS
6	NULL_DEREF	Write to address 0x10	0, 1, 2	MEM_ACCESS
7	PERIPH_CORRUPT	Overwrite one configuration register	9–12	PERIPH
8	UART_CORRUPT	Frame with a wrong checksum into the receive path	12	COMM
9	WDG_HANG	Interrupts disabled, endless loop	0, 1, 2	WATCHDOG
10	SEQ_REPEAT	Stage C executed three times	0	SEQUENCE

9.3 TASK FAULTS

Task faults are designed so that the compiler cannot remove them and they reliably trigger the protection mechanisms. For example, the stack-overflow fault uses a 64-byte volatile array at every recursion level and accesses it after the recursive call. This prevents tail-call optimisation, which could otherwise reuse the same stack frame and avoid overflow. Since each 64-byte frame is larger than the 32-byte stack guard, the guard cannot be skipped.

Listing 9.1: Stack-overflow and hang faults

```
// Stack overflow: every call uses 64 more bytes of stack
static uint32_t do_stack_overflow(uint32_t depth)
{
    volatile uint8_t buf[64];                    // 64 bytes per call
    buf[0] = (uint8_t)depth;
    return do_stack_overflow(depth + 1) + buf[0];  // never returns
}

// Stuck task: endless loop, interrupts stay enabled
for (;;) { }

// Total hang: interrupts off, only the watchdog can recover
__disable_irq();  for (;;) { }
```

The stuck-loop fault keeps interrupts enabled, causing only the target task to hang. The higher-priority monitor task can still run. In contrast, the watchdog-hang fault disables all interrupts, preventing all tasks, including the monitor, from running. Recovery is therefore possible only through the independent watchdog.

9.4 HARDWARE FAULTS

Hardware faults modify actual peripheral registers from the TIM6 interrupt. These include the ADC1 trigger-selection bits, the TIM2 prescaler (which makes sampling approximately nine times faster), the PA0 mode bits, and the USART2 sampling mode. The interrupt-storm fault changes the TIM6 prescaler to generate interrupts at 10 kHz and adds extra processing to each interrupt.

The UART fault inserts the frame $T 55*00 into the receive queue as if it had arrived through the UART. This frame requests an alarm limit of 55 mV but contains an incorrect checksum; the correct checksum is 13.

The W command writes a specified value to any memory address. It was used to corrupt a register without using the fault injector, so the monitor had no injection record for this fault.

Listing 9.2: Register corruption from the TIM6 interrupt

```
// called from the TIM6 interrupt when a hardware fault is due
void app_corrupt_periph(uint8_t comp)
{
    // ADC1: select the wrong trigger source
    if (comp == COMP_PER_ADC)  ADC1->CR2 ^= ADC_CR2_EXTSEL_0;
    // TIM2: prescaler 8 makes sampling about 9x faster
    if (comp == COMP_PER_TIM)  TIM2->PSC = 8;
    // GPIOA: PA0 is no longer an analog input
    if (comp == COMP_PER_GPIO) GPIOA->MODER &= ~3U;
    // USART2: change the bit-sampling mode
    if (comp == COMP_PER_UART) USART2->CR3 |= USART_CR3_ONEBIT;
}
```

CHAPTER 10
REAL-TIME CLOCK

During the initial tests, it was observed that fault detection and recovery usually take only 0.1 to 0.3 seconds. Although this quick recovery is beneficial, it makes the actual impact of each fault difficult to observe. By the time the LED or telemetry output is checked, the system has usually returned to normal operation. To measure the effect of each fault on the application, a real-time clock add-on was developed.

10.1 DESIGN

The clock measures application time by counting the 10 ms sample blocks received through DMA. It uses the block sequence counter already maintained by the ADC interrupt and compares this time with the independent FreeRTOS system tick. As shown in Figure 10.1, it reports two values:

Drift: The difference between the clock time and the reference time. Negative drift indicates lost samples, while positive drift indicates that samples are arriving faster than expected.

Missed deadlines: The number of 10 ms periods during which the clock task could not run because a higher-priority task was using the CPU.

A heartbeat LED connected to PA6 (D12) toggles every 0.5 seconds. It therefore blinks at 1 Hz during normal operation and stops when the system freezes.

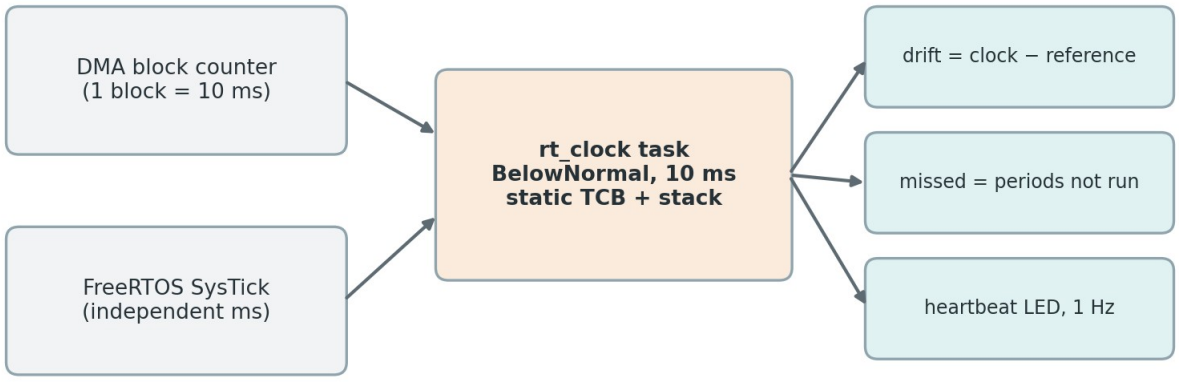


Figure 10.1: Inputs and outputs of the real-time clock add-on

The clock prints a status line every second. For example: CLK 00:01:23 | ref 00:01:23 | drift +0 ms | missed 0. It also prints an immediate CLK! message when a deadline is missed or

when the drift changes by 30 ms or more. Under normal conditions, the expected timing variation is approximately ±1 sample block. The Z command displays the clock status, while Z R resets the drift and missed-deadline counters before each test.

The add-on was implemented in a single source file. It starts automatically through a GCC constructor, which is placed in the .init_array section and called by the startup code before main(). The clock task uses a statically allocated task control block and stack, so it does not consume memory from the FreeRTOS heap used by the other tasks. It runs at BelowNormal priority and is not registered as a monitored component. Faults cannot target this task; it only observes their effects. Its variables are also kept out of the .data section to prevent changes to the address of the MPU-protected configuration block.

CHAPTER 11
TESTING METHODOLOGY

All the testing activities were carried out using the STM32 NUCLEO-F446RE development board running the FreeRTOS-based fault-monitoring application. The sensor input pin (PA0) was connected to GND to provide a stable input condition during testing, except for tests that required a different input. A heartbeat LED was connected to pin D12 through a 330 Ω current-limiting resistor. This LED was used to observe the normal operation of the application and to check whether the system continued running after a fault was detected and recovered.

The board was connected to a computer through the ST-LINK Virtual COM Port (VCP) for sending commands and monitoring system responses. The UART communication was configured with a baud rate of 115200, 8 data bits, no parity, and 1 stop bit. A serial terminal was used to communicate with the board, inject different types of faults, and observe the responses generated by the monitoring system.

For proper documentation and analysis, the output of every test was saved in a separate log file. These logs contained information about the injected faults, detection messages, localization results, recovery actions, and system timing information. Keeping separate log files makes it easier to compare the results and verify the performance of the fault-monitoring framework.

Most of the tests were performed without an active debugger session. This was important because the debugger can pause the processor when a fault exception occurs and may also freeze the watchdog timer. Such behavior could affect the actual execution of the firmware and produce results that differ from normal standalone operation. Therefore, running the board without a debugger helped verify that the monitoring and recovery mechanisms worked independently using only the microcontroller's internal resources.

11.1 PROCEDURE

A common testing procedure was followed for every fault injection to maintain consistency and ensure that the results could be compared reliably. Before starting each test, the serial terminal was cleared to avoid confusion with the output from previous tests. The clock resynchronization command (Z R) was entered first to resynchronize the real-time clock add-on. After synchronization, the required fault command was sent through the UART interface to introduce the selected fault into the application. The monitoring system then observed the

system behaviour, detected any abnormal condition, identified the possible source of the fault, and performed the required recovery action.

Once the detection and recovery process was completed, the R command was used to display the stored fault log, while the Z command was used to check the clock status. The complete terminal output was then saved as a separate log file for further analysis and verification. A minimum interval of 11 seconds was maintained between two fault injections affecting the same component. This delay was necessary to allow the monitoring system to complete its detection and recovery process before another fault was introduced. It also prevented the fault escalation mechanism from treating two closely spaced faults as a single continuous failure. Tests that could cause a complete system reset were performed at the end to avoid interrupting the remaining test activities.

Each fault test was considered successful only when all four verification conditions were satisfied:

- 1. Fault Injection: The injected fault had to be confirmed through the !GT message or the G log entry. This verified that the intended fault was introduced into the correct component.
- 2. Fault Detection: The monitoring system had to identify the abnormal condition and generate a !FAULT message. This confirmed that the fault-monitoring mechanism had successfully detected the problem.
- 3. Fault Localization: The !GT message had to display the status LOCALIZED CORRECTLY. This verified that the system had identified the actual faulty component rather than simply detecting an abnormal condition.
- 4. Fault Recovery: The system had to report the expected recovery action, such as restarting the affected task, reinitializing a peripheral, or resetting the microcontroller. The application was also required to continue operating after recovery, wherever the selected recovery action allowed it.

By following these four conditions, each test was evaluated not only on whether the fault was detected, but also on whether the system could correctly identify its source and recover from it. This provided a more complete assessment of the proposed autonomous firmware fault-localization framework.

11.2 TEST CASES

Table 11.1: Test cases of the campaign

Test	Purpose	Input
1	Boot and system health	Reset, Y 0, Z
2	Baseline learning	C, L 100, B
3	Clean run, false-alarm check	Z R, 2 minutes, Z, S
4	Application input	PA0 GND → 3.3 V → GND
5	Stuck task	I 1 0
6	ADC1 trigger corrupted	I 7 9 1000
7	TIM2 prescaler corrupted	I 7 10 1000
8	Register write without injector	W 40000028 8
9	Missed deadline	I 3 0 1000
10	Wrong execution sequence	I 10 0 1000
11	Interrupt storm	I 4 6 1000
12	Corrupted command frame	I 8 12 1000
13	Write to protected memory	I 5 0 1000
14	Stack overflow	I 2 0 1000
15	Total hang, watchdog reset	PA0 3.3 V, I 9 0 1000, PA0 to GND
16	Rule-based vs statistical localizer	M 1, I 1 0 1000, M 0
17	Cancelling a pending fault	I 1 0 5000, X
18	Fault-log readback	R, R 16, G, S, D

CHAPTER 12
RESULTS

This chapter presents the serial output recorded on the NUCLEO-F446RE for each test cases. The excerpts are taken from the log files saved during testing. Only repeated clock lines before and after the relevant part have been left out.

12.1 TEST 1: BOOT AND SYSTEM HEALTH

```
=== Fault Monitor v2 | sensor monitor | core 84 MHz | monitor ON ===
type H for commands
BOOT #4 reset cause: reset button (0x02)
black box: 348 stored reports; baseline: loaded from flash
rt_clock: real-time sample clock started (Z for status, heartbeat on D12)
TEL,9,1043,2000,0,0*4C
MONITOR running (localizer: rule)
CLK 00:00:01 | ref 00:00:01 | drift +0 ms | missed 0
TEL,19,2043,2000,0,0*7E
CLK 00:00:02 | ref 00:00:02 | drift +0 ms | missed 0
TEL,29,3043,2000,0,0*7C
CLK 00:00:03 | ref 00:00:03 | drift +0 ms | missed 0
TEL,39,4043,2000,0,0*7A
```

Figure 12.1: Serial output of Test 1

After the reset the firmware reported the cause of the restart (reset button, flag 0x02), loaded the stored baseline and fault log from flash and started the real-time clock. Monitoring began after the one-second warm-up. The telemetry frames arrive exactly 1000 ms apart (1043, 2043, 3043 ms) with the input at 0 mV, and the clock lines show zero drift and no missed deadlines. The system is healthy before any fault is injected.

12.2 TEST 2: BASELINE LEARNING

```
L 100
OK learning will start at the next window
LEARN start: 100 windows (10 s) of normal operation -- please do not inject faults now
CLK 00:29:39 | ref 00:29:39 | drift +0 ms | missed 0
CLK 00:29:40 | ref 00:29:40 | drift +0 ms | missed 0
CLK 00:29:41 | ref 00:29:41 | drift +0 ms | missed 0
CLK 00:29:42 | ref 00:29:42 | drift +0 ms | missed 0
CLK 00:29:43 | ref 00:29:43 | drift +0 ms | missed 0
CLK 00:29:44 | ref 00:29:44 | drift +0 ms | missed 0
CLK 00:29:45 | ref 00:29:45 | drift +0 ms | missed 0
CLK 00:29:46 | ref 00:29:46 | drift +0 ms | missed 0
CLK 00:29:47 | ref 00:29:47 | drift +0 ms | missed 0
CLK 00:29:48 | ref 00:29:48 | drift +0 ms | missed 0
LEARN done: baseline from 100 windows saved to flash sector 6. Monitoring.
```

Figure 12.2: Serial output of Test 2 (a)

```
B
baseline: 100 windows, core 84 MHz, crc 43511043
 0 sensor   runs=10.0 sd=0.0 cpu_us=476.6 max_slice_us=48.2 max_gap_ms=10 stack_min=326 periodic
 1 report   runs=1.0 sd=0.0 cpu_us=80.9 max_slice_us=81.1 max_gap_ms=100 stack_min=278 periodic
 2 shell    runs=1.0 sd=0.0 cpu_us=42.4 max_slice_us=42.5 max_gap_ms=100 stack_min=256 periodic
 3 monitor  runs=1.0 sd=0.0 cpu_us=985.2 max_slice_us=995.1 max_gap_ms=100 stack_min=806 periodic
 4 wdg      runs=0.4 sd=0.5 cpu_us=8.4 max_slice_us=20.9 max_gap_ms=250 stack_min=142 periodic
 5 adc_isr  runs=10.0 sd=0.0 cpu_us=155.5 max_slice_us=15.7 max_gap_ms=10 stack_min=65535 periodic
 6 tim6_isr runs=99.9 sd=0.8 cpu_us=753.4 max_slice_us=8.0 max_gap_ms=1 stack_min=65535 periodic
 7 uart_isr runs=5.5 sd=16.6 cpu_us=33.0 max_slice_us=18.5 max_gap_ms=995 stack_min=65535 event-driven
 9 ADC1     cfg_crc=f19ba914
10 TIM2     cfg_crc=9458df35
11 GPIOA    cfg_crc=bc92671d
12 USART2   cfg_crc=e6dd9d35
allowed stage transitions: A>B B>C C>D D>A E>E
```

Figure 12.3: Serial output of Test 2 (b)

Learning started at a window boundary and finished exactly 10 s later, after 100 windows of 100 ms, and the baseline was saved to flash sector 6. The B command shows what was learned: the sensor task runs 10 times per window with zero variation and about 48 µs per run, the ADC interrupt runs 10 times at under 16 µs, and the 1 kHz timer interrupt about 100 times. The configuration checksums of ADC1, TIM2, GPIOA and USART2 were recorded, and the allowed stage order is A>B>C>D. The monitor itself needs about 1 ms per window, roughly 1 % of the CPU.

12.3 TEST 3: CLEAN RUN (FALSE-ALARM CHECK)

```
Z R
OK clock resync at the next period
CLK 00:00:00 | ref 00:00:00 | drift +0 ms | missed 0
CLK 00:00:01 | ref 00:00:01 | drift +0 ms | missed 0
CLK 00:00:02 | ref 00:00:02 | drift +0 ms | missed 0
CLK 00:00:03 | ref 00:00:03 | drift +0 ms | missed 0
```

Figure 12.4: Serial output of Test 3 (a)

```
CLK 00:02:20 | ref 00:02:20 | drift +0 ms | missed 0
S
monitor: monitoring | localizer rule | faults 0 | recoveries 0 | stored 0 | boot #3
window 7532  (100000 us)  core 84 MHz  heap free 16504 B
id name        kind runs  cpu_us  max_us  gap_us  stack_free
 0 sensor     task    10     476      47   10000    326
 1 report     task     1      79      79  100000    278
 2 shell      task     1      43      43   99999    250
 3 monitor    task     1    1225    1225  100000    806
 4 wdg        task     0       0       0  211004    142
 5 adc_isr     isr    10     155      15   10000  65535
 6 tim6_isr    isr   100     753       7    1000  65535
 9 ADC1        per cfg_crc=f19ba914 errors=0
10 TIM2        per cfg_crc=9458df35 errors=0
11 GPIOA       per cfg_crc=bc92671d errors=0
12 USART2      per cfg_crc=e6dd9d35 errors=0
ring: 1023 events held, 1166076 overwritten
OVH,cpu_monitor=1.22%,ram_framework=15404 B,trace_storage=8204 B,report=164 B
```

Figure 12.5: Serial output of Test 3 (b)

After the clock was resynchronised, the board was left running for 2 minutes 20 seconds, about 1 300 monitoring windows, without any input. No fault was reported and the clock stayed at zero drift. The status output shows faults 0 and recoveries 0, the live timing of every component matches the baseline, and all four peripheral checksums are unchanged. The overhead line gives the measured cost of the monitor: 1.22 % of the CPU, 15 404 bytes of RAM for the framework and 8 204 bytes for the trace buffer.

12.4 TEST 4: APPLICATION INPUT (PA0 AT GND AND 3.3 V)

```
TEL,11389,1138900,2000,919,0*72
CLK 00:00:25 | ref 00:00:25 | drift +0 ms | missed 0
TEL,11399,1139900,2000,921,0*79
CLK 00:00:26 | ref 00:00:26 | drift +0 ms | missed 0
TEL,11409,1140900,2000,911,0*7A
CLK 00:00:27 | ref 00:00:27 | drift +0 ms | missed 0
TEL,11419,1141900,2000,874,0*78
CLK 00:00:28 | ref 00:00:28 | drift +0 ms | missed 0
TEL,11429,1142900,2000,1344,0*41
CLK 00:00:29 | ref 00:00:29 | drift +0 ms | missed 0
TEL,11439,1143900,2000,3299,1*43
CLK 00:00:30 | ref 00:00:30 | drift +0 ms | missed 0
TEL,11449,1144900,2000,3299,1*43
CLK 00:00:31 | ref 00:00:31 | drift +0 ms | missed 0
TEL,11459,1145900,2000,3299,1*43
```

Figure 12.6: Serial output of Test 4 (a)

```
TEL,11509,1150900,2000,3300,1*42
CLK 00:00:37 | ref 00:00:37 | drift +0 ms | missed 0
TEL,11519,1151900,2000,917,0*7C
CLK 00:00:38 | ref 00:00:38 | drift +0 ms | missed 0
TEL,11529,1152900,2000,917,0*7C
CLK 00:00:39 | ref 00:00:39 | drift +0 ms | missed 0
TEL,11539,1153900,2000,917,0*7C
CLK 00:00:40 | ref 00:00:40 | drift +0 ms | missed 0
TEL,11549,1154900,2000,919,0*72
CLK 00:00:41 | ref 00:00:41 | drift +0 ms | missed 0
TEL,11559,1155900,2000,0,0*73
CLK 00:00:42 | ref 00:00:42 | drift +0 ms | missed 0
TEL,11569,1156900,2000,0,0*73
```

Figure 12.7: Serial output of Test 4 (b)

The fifth field of each TEL line is the measured input. While the wire was lifted the input floated at about 920 mV; connected to 3.3 V the reading rose through 1344 mV (the smoothing filter catching up) to 3299–3300 mV, above the 2000 mV limit, so the alarm field changed to 1 and LED LD2 switched on. Back at GND the reading fell to 0 mV and the alarm switched off. No fault was reported at any point and the clock kept zero drift, because a change of input is normal behaviour, not a firmware fault.

12.5 TEST 5: STUCK TASK

```
I 1 0
OK armed STUCK_LOOP on sensor in 0 ms; expect: EXEC_OVERRUN, restart task
!SHADOW stat says: sensor EXEC_OVERRUN conf=89%

!FAULT seq=1 type=EXEC_OVERRUN comp=sensor(0) conf=99% loc=rule action=RESTART_TASK rules=0x0003
pc=0x0800611e uptime=3011916ms latency_ms=97
!HIST   -27892 us  adc_isr  ISR_ENTER 0
!HIST   -27880 us  adc_isr  ISR_EXIT 12
!HIST   -17892 us  adc_isr  ISR_ENTER 0
!HIST   -17880 us  adc_isr  ISR_EXIT 12
!HIST    -7892 us  adc_isr  ISR_ENTER 0
!HIST    -7880 us  adc_isr  ISR_EXIT 12
!HIST    -5624 us  sensor   TASK_OUT 0
!HIST    -5616 us  monitor  TASK_IN 0
!HIST    -4285 us  monitor  TASK_OUT 0
!HIST    -4274 us  sensor   TASK_IN 0
!HIST      -38 us  sensor   TASK_OUT 0
!HIST      -30 us  monitor  TASK_IN 0
!HIST        7 us  sensor   FAULT 1
!GT injected=1 on sensor -> LOCALIZED CORRECTLY
!RECOVERED comp=sensor action=RESTART_TASK level=0
CLK! missed 16 deadline(s): task blocked ~167 ms
CLK 00:00:20 | ref 00:00:20 | drift +3 ms | missed 16
```

Figure 12.8: Serial output of Test 5

An infinite loop was injected into the sensor task. The history shows the ADC interrupt still arriving every 10 ms, while the sensor task, switched in at −4274 µs, never reaches its first processing stage. At the next 100 ms check, 97 ms after the fault, the monitor reported an execution overrun on the sensor task with 99 % confidence; the statistical localizer agreed at 89 %. The task was deleted and recreated without a reset. The real-time clock shows the damage to the application: it was blocked for 167 ms and missed 16 of its 10 ms deadlines.

12.6 TEST 6: ADC TRIGGER CORRUPTED

```
I 7 9 1000
OK armed PERIPH_CORRUPT on ADC1 in 1000 ms; expect: PERIPH config CRC, reinit peripheral
CLK 00:00:18 | ref 00:00:18 | drift +0 ms | missed 0
CLK! drift -30 ms: sample blocks lost (sampling stopped)
!SHADOW stat says: ADC1 PERIPH conf=95%

!FAULT seq=2 type=PERIPH comp=ADC1(9) conf=95% loc=rule action=REINIT_PERIPH rules=0x0062 pc=0x00000000
uptime=175215ms latency_ms=99
!HIST    -7822 us  sensor   TASK_IN 0
!HIST    -7811 us  ADC1     PERIPH_ERR 8
!HIST    -7807 us  sensor   STAGE A_acquire
!HIST    -7798 us  sensor   STAGE B_filter
!HIST    -7788 us  sensor   STAGE C_compare
!HIST    -7782 us  sensor   STAGE D_actuate
!HIST    -7769 us  sensor   TASK_OUT 0
!HIST        2 us  ADC1     FAULT 8
!GT injected=7 on ADC1 -> LOCALIZED CORRECTLY
!RECOVERED comp=ADC1 action=REINIT_PERIPH level=0
CLK! drift -180 ms: sample blocks lost (sampling stopped)
CLK 00:00:19 | ref 00:00:19 | drift -180 ms | missed 0
CLK 00:00:20 | ref 00:00:20 | drift -180 ms | missed 0
```

Figure 12.9: Serial output of Test 6

The trigger-selection bits of ADC1 were overwritten, so the converter stopped following its 1 kHz timer and sampling stopped. The sensor task logged a 50 ms timeout against ADC1

(PERIPH_ERR 8) and the clock reported the first missing samples. 99 ms after the fault the monitor blamed ADC1 with 95 % confidence: its configuration checksum no longer matched the baseline (rules 0x0062). ADC1 was reset and reconfigured. The clock settled at −180 ms and stayed there, which means 18 sample blocks were lost and sampling was fully restored.

12.7 TEST 7: SAMPLING TIMER CORRUPTED

```
I 7 10 1000
OK armed PERIPH_CORRUPT on TIM2 in 1000 ms; expect: PERIPH config CRC, reinit peripheral
CLK 00:00:13 | ref 00:00:13 | drift +0 ms | missed 0
CLK! drift +80 ms: sample blocks arriving too fast
!SHADOW stat says: TIM2 PERIPH conf=95%

!FAULT seq=3 type=PERIPH comp=TIM2(10) conf=95% loc=rule action=REINIT_PERIPH rules=0x0024 pc=0x00000000
 uptime=536815ms latency_ms=79
!HIST    -1340 us  sensor   STAGE D_actuate
!HIST    -1328 us  sensor   TASK_OUT 0
!HIST     -345 us  adc_isr  ISR_ENTER 0
!HIST     -329 us  adc_isr  ISR_EXIT 15
!HIST     -315 us  sensor   TASK_IN 0
!HIST     -294 us  sensor   STAGE A_acquire
!HIST     -265 us  sensor   STAGE D_actuate
!HIST        2 us  TIM2     FAULT 8
!GT injected=7 on TIM2 -> LOCALIZED CORRECTLY
!RECOVERED comp=TIM2 action=REINIT_PERIPH level=0
CLK! drift +1300 ms: sample blocks arriving too fast
CLK 00:00:15 | ref 00:00:14 | drift +1300 ms | missed 0
```

Figure 12.10: Serial output of Test 7

The prescaler of TIM2, the timer that triggers the ADC, was overwritten, so sampling ran about nine times too fast; the history shows complete pipeline cycles about 1 ms apart instead of 10 ms. The monitor blamed TIM2 with 95 % confidence after 79 ms. Two rules fired (0x0024): the changed TIM2 checksum, which is the cause, and the higher interrupt rate, which is only the symptom. TIM2 was restored to 1 kHz, and the clock ended 1.3 s ahead of real time, the timing error the application suffered.

12.8 TEST 8: REGISTER WRITE WITHOUT THE INJECTOR

```
W 40000028 8
OK writing 0x00000008 to 0x40000028
CLK! drift +80 ms: sample blocks arriving too fast
!SHADOW stat says: TIM2 PERIPH conf=95%

!FAULT seq=4 type=PERIPH comp=TIM2(10) conf=95% loc=rule action=REINIT_PERIPH rules=0x0024 pc=0x00000000
 uptime=1007315ms latency_ms=-1
!HIST    -1778 us  sensor   STAGE D_actuate
!HIST    -1766 us  sensor   TASK_OUT 0
!HIST     -785 us  adc_isr  ISR_ENTER 0
!HIST     -770 us  adc_isr  ISR_EXIT 15
!HIST     -756 us  sensor   TASK_IN 0
!HIST     -709 us  sensor   STAGE D_actuate
!HIST        2 us  TIM2     FAULT 8
!RECOVERED comp=TIM2 action=REINIT_PERIPH level=0
CLK! drift +1350 ms: sample blocks arriving too fast
CLK 00:00:16 | ref 00:00:15 | drift +1350 ms | missed 0
```

Figure 12.11: Serial output of Test 8

Here the TIM2 prescaler was written directly to address 0x40000028, without the fault injector, so the monitor had no injection record and no hint. It still detected the changed register and blamed TIM2 with 95 % confidence, then restored the timer. There is no !GT line and the latency is −1 because nothing was injected that could be compared or timed. The clock gained 1350 ms, almost the same as in the injected version, which shows that detection does not depend on how the fault was caused.

12.9 TEST 9: MISSED DEADLINE

```
I 3 0 1000
OK armed MISSED_DEADLINE on sensor in 1000 ms; expect: LIVENESS, restart task
CLK 00:00:12 | ref 00:00:12 | drift +0 ms | missed 0
!SHADOW stat says: sensor LIVENESS conf=97%

!FAULT seq=5 type=LIVENESS comp=sensor(0) conf=87% loc=rule action=RESTART_TASK rules=0x0002
pc=0x0800f212 uptime=1193416ms latency_ms=172
!HIST    -2797 us  sensor   TASK_IN 0
!HIST    -2786 us  sensor   STAGE A_acquire
!HIST    -2777 us  sensor   STAGE B_filter
!HIST    -2767 us  sensor   STAGE C_compare
!HIST    -2761 us  sensor   STAGE D_actuate
!HIST    -2738 us  sensor   TASK_OUT 0
!HIST        6 us  sensor   FAULT 3
!GT injected=3 on sensor -> LOCALIZED CORRECTLY
!RECOVERED comp=sensor action=RESTART_TASK level=0
CLK 00:00:13 | ref 00:00:13 | drift +0 ms | missed 0
CLK 00:00:14 | ref 00:00:14 | drift +0 ms | missed 0
```

Figure 12.12: Serial output of Test 9

The sensor task was made to sleep for 100 ms instead of 10 ms. Its learned limit for silence is 22 ms (twice the normal 10 ms gap plus 2 ms), so a 100 ms gap is about 4.5 times over the limit, and the monitor reported a liveness fault on the sensor task with 87 % confidence ($55 + 15 \cdot \log_2 4.5$). The latency of 172 ms is the 100 ms of oversleeping plus the time to the next window check. The clock shows no change, because the hardware kept sampling and the CPU stayed free: this fault is visible only to the monitor.

12.10 TEST 10: WRONG EXECUTION SEQUENCE

```
I 10 0 1000
OK armed SEQ_REPEAT on sensor in 1000 ms; expect: SEQUENCE (stage out of order), restart task 0
CLK 00:00:14 | ref 00:00:14 | drift +0 ms | missed 0
!SHADOW stat says: sensor SEQUENCE conf=90%

!FAULT seq=6 type=SEQUENCE comp=sensor(0) conf=90% loc=rule action=RESTART_TASK rules=0x0010
pc=0x0800f212 uptime=1521416ms latency_ms=72
!HIST    -2710 us  sensor   TASK_IN 0
!HIST    -2700 us  sensor   STAGE A_acquire
!HIST    -2691 us  sensor   STAGE B_filter
!HIST    -2680 us  sensor   STAGE C_compare
!HIST    -2675 us  sensor   STAGE D_actuate
!HIST    -2652 us  sensor   TASK_OUT 0
!HIST        6 us  sensor   FAULT 4
!GT injected=10 on sensor -> LOCALIZED CORRECTLY
!RECOVERED comp=sensor action=RESTART_TASK level=0
CLK 00:00:15 | ref 00:00:15 | drift +0 ms | missed 0
```

Figure 12.13: Serial output of Test 10

On one cycle the sensor pipeline executed its compare stage three times in a row (A → B → C → C → C → D). The transition C → C never appeared during learning, so the monitor treated it as hard evidence of a control-flow fault and reported it 72 ms later with 90 % confidence, then restarted the task. The repeated stages are no longer in the 16-event history because they happened earlier in the window; the history shows the next, normal cycle. The clock was unaffected: the fault was caught before it changed the application's timing or output.

12.11 TEST 11: INTERRUPT STORM

```
I 4 6 1000
OK armed ISR_STORM on tim6_isr in 1000 ms; expect: ISR_RATE on tim6_isr, reinit TIM6
CLK 00:00:12 | ref 00:00:12 | drift +0 ms | missed 0
!SHADOW stat says: tim6_isr ISR_RATE conf=79%

!FAULT seq=7 type=ISR_RATE comp=tim6_isr(6) conf=99% loc=rule action=REINIT_PERIPH rules=0x0004
pc=0x00000000 uptime=1916218ms latency_ms=52
!HIST    -4713 us  adc_isr  ISR_ENTER 0
!HIST    -4698 us  adc_isr  ISR_EXIT 15
!HIST    -4081 us  sensor   TASK_IN 0
!HIST    -4070 us  sensor   STAGE A_acquire
!HIST    -3995 us  sensor   STAGE B_filter
!HIST    -3985 us  sensor   STAGE C_compare
!HIST    -3979 us  sensor   STAGE D_actuate
!HIST    -3901 us  sensor   TASK_OUT 0
!GT injected=4 on tim6_isr -> LOCALIZED CORRECTLY
!RECOVERED comp=tim6_isr action=REINIT_PERIPH level=0
CLK 00:00:13 | ref 00:00:13 | drift +0 ms | missed 0
```

Figure 12.14: Serial output of Test 11

The TIM6 interrupt was switched from 1 kHz to 10 kHz with extra work in each interrupt. Its side effect is visible in the history: the sensor task's stages are stretched from about 10 µs to 75 µs apart because the interrupts keep preempting it. The monitor saw TIM6 firing far more often than the 99.9 ± 0.8 times per window it had learned and reported an interrupt-rate fault with 99 % confidence after only 52 ms. TIM6 was set back to 1 kHz and the clock showed no damage.

12.12 TEST 12: CORRUPTED COMMAND FRAME

```
I 8 12 1000
OK armed UART_CORRUPT on USART2 in 1000 ms; expect: COMM frame checksum, reinit USART2
CLK 00:00:12 | ref 00:00:12 | drift +0 ms | missed 0
$T 55*00
ERR frame checksum
!SHADOW stat says: USART2 COMM conf=90%

!FAULT seq=8 type=COMM comp=USART2(12) conf=90% loc=rule action=REINIT_PERIPH rules=0x0084 pc=0x00000000
 uptime=2121415ms latency_ms=62
!HIST        2 us  USART2   FAULT 9
!GT injected=8 on USART2 -> LOCALIZED CORRECTLY
!RECOVERED comp=USART2 action=REINIT_PERIPH level=0
CLK 00:00:13 | ref 00:00:13 | drift +0 ms | missed 0
```

Figure 12.15: Serial output of Test 12 (a)

```
Y 1
OK telemetry on
CLK 00:00:27 | ref 00:00:27 | drift +0 ms | missed 0
TEL,21359,2135900,2000,0,0*73
CLK 00:00:28 | ref 00:00:28 | drift +0 ms | missed 0
TEL,21369,2136900,2000,0,0*73
```

Figure 12.16: Serial output of Test 12 (b)

A command to set the alarm limit to 55 mV arrived with a wrong checksum, as if the serial line had been corrupted. The shell rejected it (ERR frame checksum), and the monitor reported a communication fault on USART2 with 90 % confidence after 62 ms and re-initialised the UART. The telemetry afterwards still shows the 2000 mV limit, so the corrupted command had no effect on the application.

12.13 TEST 13: WRITE TO PROTECTED MEMORY

```
I 5 0 1000
OK armed MEM_RO_WRITE on sensor in 1000 ms; expect: MEM_ACCESS (MPU read-only), reset + report
CLK 00:00:14 | ref 00:00:14 | drift +0 ms | missed 0

=== Fault Monitor v2 | sensor monitor | core 84 MHz | monitor ON ===
type H for commands
BOOT #3 reset cause: software reset (0x06)
!TRAP before reset: trap=2 cfsr=0x00000082 hfsr=0x00000000 addr=0x20000000 lr=0x080061f5

!FAULT seq=9 type=MEM_ACCESS comp=sensor(0) conf=95% loc=rule action=SYSTEM_RESET rules=0x0100
pc=0x080061f8 uptime=0ms latency_ms=-1
!HIST      -64 us  adc_isr  ISR_ENTER 0
!HIST      -35 us  adc_isr  ISR_EXIT 28
!HIST      -27 us  other    TASK_OUT 0
!HIST      -21 us  sensor   TASK_IN 0
!HIST       -9 us  sensor   INJECT 5
!HIST        0 us  sensor   MEM_FAULT 130
!GT injected fault 5 on comp 0 before the reset, trap latency 9 us
black box: 9 stored reports; baseline: loaded from flash
```

Figure 12.17: Serial output of Test 13

The fault tried to overwrite the application's configuration block, which the memory protection unit keeps read-only. The write was blocked by the hardware and raised a MemManage fault; the board saved a crash record and restarted itself (software reset). After the reboot the report shows CFSR 0x82, a data-access violation with a valid address, and the exact

address 0x20000000 of the protected block. The fault was attributed to the sensor task with 95 % confidence and was trapped 9 µs after the faulty write. All earlier reports were still in flash.

12.14 TEST 14: STACK OVERFLOW

```
I 2 0 1000
OK armed STACK_OVERFLOW on sensor in 1000 ms; expect: STACK_OVERFLOW via MPU guard, reset + report
CLK 00:00:12 | ref 00:00:12 | drift +0 ms | missed 0

=== Fault Monitor v2 | sensor monitor | core 84 MHz | monitor ON ===
type H for commands
BOOT #3 reset cause: software reset (0x06)
!TRAP before reset: trap=2 cfsr=0x00000082 hfsr=0x00000000 addr=0x20003764 lr=0x08006155

!FAULT seq=10 type=STACK_OVERFLOW comp=sensor(0) conf=99% loc=rule action=SYSTEM_RESET rules=0x0100
pc=0x08006126 uptime=0ms latency_ms=-1
!HIST     -281 us  adc_isr  ISR_ENTER 0
!HIST     -265 us  adc_isr  ISR_EXIT 15
!HIST     -257 us  other    TASK_OUT 0
!HIST     -251 us  sensor   TASK_IN 0
!HIST     -239 us  sensor   INJECT 2
!HIST        0 us  sensor   STACK_FAULT 130
!GT injected fault 2 on comp 0 before the reset, trap latency 239 us
black box: 10 stored reports; baseline: loaded from flash
```

Figure 12.18: Serial output of Test 14

Unbounded recursion was started inside the sensor task, using 64 bytes of stack per call. After 239 µs the stack reached the 32-byte no-access guard below it, and the first write into the guard was stopped by the MPU before any other memory was corrupted. Because the faulting address 0x20003764 lies inside that guard, the fault was classified as a stack overflow with 99 % confidence. The board reset itself and the report was filed after the reboot.

12.15 TEST 15: TOTAL HANG AND WATCHDOG RESET

```
TEL,1949,194904,2000,3299,1*47
I 9 0 1000
OK armed WDG_HANG on sensor in 1000 ms; expect: WATCHDOG reset (~4 s), report after reboot
CLK 00:00:17 | ref 00:00:17 | drift +0 ms | missed 0
TEL,1959,195904,2000,3299,1*47

=== Fault Monitor v2 | sensor monitor | core 84 MHz | monitor ON ===
type H for commands
BOOT #3 reset cause: IWDG watchdog (0x0a)
!WATCHDOG reset: the CPU stopped serving the monitor; last running: sensor, last stage: 3

!FAULT seq=12 type=WATCHDOG comp=sensor(0) conf=80% loc=rule action=SYSTEM_RESET rules=0x0200
pc=0x00000000 uptime=0ms latency_ms=-1
!HIST      -41 us  adc_isr  ISR_ENTER 0
!HIST      -25 us  adc_isr  ISR_EXIT 15
!HIST      -18 us  other    TASK_OUT 0
!HIST      -11 us  sensor   TASK_IN 0
!HIST        0 us  sensor   INJECT 9
!GT injected fault 9 on comp 0 before the reset
black box: 12 stored reports; baseline: loaded from flash
rt_clock: real-time sample clock started (Z for status, heartbeat on D12)
TEL,9,904,2000,0,0*77
```

Figure 12.19: Serial output of Test 15

Before the fault the input was at 3.3 V and the alarm was on (3299 mV, alarm 1). The fault disabled all interrupts and looped forever, so nothing could run, not even the monitor, and the output stopped. The input was moved to 0 V during the freeze, but the frozen application could not switch the alarm off, and the heartbeat LED stopped. About 4 s later the independent watchdog reset the chip. After the reboot the reset cause read IWDG watchdog and the monitor named the sensor task, the task that was running when the system froze, with 80 % confidence. The first telemetry line shows 0 mV with the alarm off: the application was following its input again.

12.16 TEST 16: RULE-BASED VS STATISTICAL LOCALIZER

```
M 1
OK localizer: stat (the other runs in shadow mode)
!SHADOW rule says: - NONE conf=0%

!FAULT seq=13 type=EXEC_OVERRUN comp=shell(2) conf=98% loc=stat action=RESTART_TASK rules=0x0400
pc=0x0800c9f8 uptime=340005ms latency_ms=-1
!RECOVERED comp=shell action=RESTART_TASK level=0
```

Figure 12.20: Serial output of Test 16 (a)

```
I 1 0 1000
OK armed STUCK_LOOP on sensor in 1000 ms; expect: EXEC_OVERRUN, restart task
!SHADOW rule says: sensor EXEC_OVERRUN conf=99%

!FAULT seq=16 type=EXEC_OVERRUN comp=sensor(0) conf=99% loc=stat action=RESTART_TASK rules=0x0400
pc=0x080061e8 uptime=361905ms latency_ms=31
!HIST   -31143 us  sensor   TASK_IN 0
!HIST   -31131 us  sensor   INJECT 1
!HIST   -21173 us  adc_isr  ISR_ENTER 0
!HIST   -21161 us  adc_isr  ISR_EXIT 12
!HIST        7 us  sensor   FAULT 1
!GT injected=1 on sensor -> LOCALIZED CORRECTLY
!RECOVERED comp=sensor action=RESTART_TASK level=0
CLK! missed 10 deadline(s): task blocked ~101 ms
M 0
OK localizer: rule (the other runs in shadow mode)
```

Figure 12.21: Serial output of Test 16 (b)

With the statistical localizer acting, the real stuck-task fault was found by both methods with 99 % confidence, 31 ms after injection. The statistical localizer, however, also raised false alarms on the command shell every time a command was typed (the first output), three in total, while the rule-based localizer reported nothing (NONE). During learning the shell showed almost no variation, so normal command handling looked like a large statistical deviation. The rule-based thresholds have absolute minimum values and are not affected, which is why the rule-based localizer is used by default.

12.17 TEST 17: CANCELLING A PENDING FAULT

```
I 1 0 5000
OK armed STUCK_LOOP on sensor in 5000 ms; expect: EXEC_OVERRUN, restart task
CLK 00:00:12 | ref 00:00:12 | drift +0 ms | missed 0
CLK 00:00:13 | ref 00:00:13 | drift +0 ms | missed 0
CLK 00:00:14 | ref 00:00:14 | drift +0 ms | missed 0
CLK 00:00:15 | ref 00:00:15 | drift +0 ms | missed 0
X
OK cancelled (the armed fault will not fire)
CLK 00:00:16 | ref 00:00:16 | drift +0 ms | missed 0
CLK 00:00:17 | ref 00:00:17 | drift +0 ms | missed 0
CLK 00:00:18 | ref 00:00:18 | drift +0 ms | missed 0
G
cycle_ts,comp_id,fault_id,delay_ms
556711617,0,1,5000
976746329,0,1,0
2479284174,0,1,5000
1117012760,0,1,5000
END
```

Figure 12.22: Serial output of Test 17

A stuck-loop fault was armed with a 5 s delay and cancelled about 4 s later. Nothing happened afterwards: no fault report and no change in the clock. The ground-truth log confirms it. Each fault writes one row when it is armed (delay 5000) and one when it fires (delay 0); the last row of this run has no fire row after it, so the fault never executed. The monitor reports only real misbehaviour, never the injection request itself.

12.18 TEST 18: FAULT LOG AFTER THE CAMPAIGN

```
R
#R,seq,uptime_ms,type,comp,conf,action,loc,rules,pc,reset_cause
R,1,3011916,EXEC_OVERRUN,sensor,99,RESTART_TASK,rule,0x0003,0x0800611e,reset button
R,2,175215,PERIPH,ADC1,95,REINIT_PERIPH,rule,0x0062,0x00000000,reset button
R,3,536815,PERIPH,TIM2,95,REINIT_PERIPH,rule,0x0024,0x00000000,reset button
R,4,1007315,PERIPH,TIM2,95,REINIT_PERIPH,rule,0x0024,0x00000000,reset button
R,5,1193416,LIVENESS,sensor,87,RESTART_TASK,rule,0x0002,0x0800f212,reset button
R,6,1521416,SEQUENCE,sensor,90,RESTART_TASK,rule,0x0010,0x0800f212,reset button
R,7,1916218,ISR_RATE,tim6_isr,99,REINIT_PERIPH,rule,0x0004,0x00000000,reset button
R,8,2121415,COMM,USART2,90,REINIT_PERIPH,rule,0x0084,0x00000000,reset button
R,9,0,MEM_ACCESS,sensor,95,SYSTEM_RESET,rule,0x0100,0x080061f8,software reset
R,10,0,STACK_OVERFLOW,sensor,99,SYSTEM_RESET,rule,0x0100,0x08006126,software reset
R,11,0,WATCHDOG,monitor,80,SYSTEM_RESET,rule,0x0200,0x00000000,IWDG watchdog
R,12,0,WATCHDOG,sensor,80,SYSTEM_RESET,rule,0x0200,0x00000000,IWDG watchdog
R,13,340005,EXEC_OVERRUN,shell,98,RESTART_TASK,stat,0x0400,0x0800c9f8,IWDG watchdog
R,14,348704,EXEC_OVERRUN,shell,98,RESTART_TASK,stat,0x0400,0x0800c9f8,IWDG watchdog
R,15,360904,EXEC_OVERRUN,shell,99,RESTART_TASK,stat,0x0400,0x0800c9f8,IWDG watchdog
R,16,361905,EXEC_OVERRUN,sensor,99,RESTART_TASK,stat,0x0400,0x080061e8,IWDG watchdog
R,17,686105,EXEC_OVERRUN,sensor,99,RESTART_TASK,rule,0x0003,0x080061ea,IWDG watchdog
R,18,73023,EXEC_OVERRUN,sensor,99,RESTART_TASK,rule,0x0001,0x080061b0,reset button
R,19,177022,COMM,USART2,90,REINIT_PERIPH,rule,0x0080,0x00000000,reset button
END 19 reports
```

Figure 12.23: Serial output of Test 18

At the end of the campaign the fault log in flash still held all 19 reports, including those written before four fault resets and three reflashes of the firmware. Each line gives the fault type, the component, the confidence, the recovery action, the localizer, the rules that fired, the program counter and the reset cause of the boot in which it was read. Report 11 is the first

watchdog run, recorded before a firmware fix (it named the monitor task); report 12 is the corrected repeat. Reports 13–15 are the statistical localizer's false alarms from Test 16, and report 19 is a genuine UART receive error that occurred during Test 17 without any injection.

After the reset the firmware reported the cause of the restart (reset button, flag 0x02), loaded the stored baseline and fault log from flash and started the real-time clock. Monitoring began after the one-second warm-up. The telemetry frames arrive exactly 1000 ms apart (1043, 2043, 3043 ms) with the input at 0 mV, and the clock lines show zero drift and no missed deadlines. The system is healthy before any fault is injected.

12.19 IMPACT ON THE APPLICATION

The clock add-on separates the faults into three groups. Faults 1, 7 on ADC1 and 7 on TIM2 damaged the application: it lost deadlines, lost samples or ran with the wrong timing. Faults that stopped the CPU (memory traps and the watchdog hang) cost the application its running state through the reset, and the hang caused a visible missed alarm. The remaining faults, the missed deadline, the sequence fault, the interrupt storm and the corrupted frame, were caught without any effect on the clock. For the missed deadline this means only that the clock could not see the damage; the monitor still detected it.

12.20 SUMMARY OF MEASURED RESULTS

Table 12.1: Key results of the test campaign

Metric	Value
Fault events detected	15 of 15, including the raw register write
Localized correctly	14 of 15 on the first attempt; the watchdog case after a fix and repeat
False alarms in the clean run	0 in about 1 300 windows
False alarms over the whole campaign	Rule-based 1, statistical 3, all on the event-driven shell
Detection latency, runtime faults	31–172 ms, median 72 ms
Detection latency, memory traps	9 µs (protected write), 239 µs (stack overflow)
Monitor CPU overhead	1.22–1.23 %
Framework RAM	15 404 B, plus 8 204 B trace buffer
Report size	164 B; 19 reports kept through 4 fault resets and 3 reflashes
Application cost	Sensor task 48 µs per 10 ms, ADC interrupt 15 µs; system about 97 % idle

CHAPTER 13
DISCUSSION

13.1 ANSWER TO THE RESEARCH QUESTION

The results support a qualified yes. Using only autonomous on-chip resources, the monitor detected every fault event in the campaign, named the right component in all but one case on the first attempt, recovered without a PC and left a report that explains what happened. The clean run produced no false alarms, and the cost was about 1.2 % of the CPU. The qualification is that the evaluation consists of one to two trials per fault class on one board, so the numbers demonstrate that the mechanisms work rather than giving statistically strong rates.

13.2 WHY HARD EVIDENCE MATTERS

The most reliable localizations came from binary evidence: a changed configuration checksum, a stage transition never seen in learning, an MPU violation with a known address. They gave 90 to 99 % confidence and never pointed to the wrong component. They were also the reason why symptoms did not mislead the monitor. When the ADC stopped, the sensor task went silent and could have been blamed for a missed deadline, but the checksum of ADC1 made the cause clear. The same happened with TIM2, where the faster interrupt rate was a symptom and the changed prescaler the cause.

13.3 TIMING EVIDENCE AND THE LOGARITHMIC SCORE

Timing evidence needs a good baseline and is graded rather than certain. The logarithmic confidence behaved as intended. A task that used about 200 times its normal CPU time was capped at 99 %, while a task silent for 4.5 times its limit received 87 %. Debouncing over two windows and victim suppression kept starved low-priority tasks from being blamed during the stuck-task fault.

13.4 RULE-BASED VERSUS STATISTICAL

On real faults the two localizers agreed. The difference appeared on an event-driven task. A statistical score measures distance in standard deviations, so a component that showed almost no variation during learning becomes very sensitive later. Absolute floors in the rule-based thresholds avoid this. For this system the rule-based localizer is the better default, and the statistical localizer would need a larger variance floor or a separate treatment of event-driven tasks.

13.5 COMPARISON WITH OTHER APPROACHES

Compared with offline techniques such as spectrum-based localization [11], [12] or post-fuzzing root-cause analysis [8], our monitor localizes at the level of components rather than statements, but it does so while the device runs and it acts on the result. Compared with a plain watchdog, which only resets, it names the component, distinguishes ten fault types, recovers most of them without a reset and keeps a history. The cost of these capabilities, about 1.2 % CPU and 23 KB of RAM including the trace buffer, is small on a microcontroller of this class.

CHAPTER 14
CHALLENGES AND LIMITATIONS

14.1 ISSUES FACED AND HOW THEY WERE RESOLVED

Several problems appeared while building and testing the system. Table 14.1 lists the main ones, their causes and the changes made.

Table 14.1: Issues faced during the project and their resolution

Issue observed	Root cause	Fix or mitigation
Some fault commands rejected with "invalid fault/target"	Each fault accepts only certain components, but the fault list did not show which	The fault list now prints the valid targets, and errors name them
History showed −51 130 562 µs for the trap event	A tiny positive cycle difference was converted as unsigned and wrapped to 2^32 cycles	Signed difference in the history printer; the event now shows 0 µs
Effects of faults invisible	Detection and recovery take 0.1–0.3s	Real-time clock add-on with drift, missed deadlines and heartbeat LED
Board froze at memory faults, watchdog never fired	A debug session halted the CPU at the fault handler and froze the watchdog	Autonomous tests run without a debugger session
Watchdog report blamed the monitor task	Last running task read after the new boot had overwritten it	Value captured before the scheduler starts; test repeated
Statistical localizer false alarms on the shell	Near-zero variance of the shell during learning	Rule-based localizer kept as default
Clock drift of +10 to +20 ms after learning	Flash writes stall the CPU and shift the block count by one block	Clock resynchronised before every test
Commands echoed but not executed	Terminal sent no line ending	Commands sent with Enter
Cancel reported success after the fault had fired	Message did not check whether a fault was pending	Message now reports the real state

14.2 LIMITATIONS

- Post-Reset Reporting: Memory faults and watchdog resets are reported only after the system restarts. A complete system hang cannot record the program counter or detection time.
- Event-Driven Tasks: The command shell produced one false alarm with the rule-based method and three with the statistical method.
- Limited Event History: Each report stores only the latest 16 events. Some earlier events may be missing, although the complete trace stores up to 1023 events.
- Execution-Based Detection: Incorrect data values that do not affect execution timing or behaviour cannot be detected.

- Limited Memory Protection: The MPU has eight regions, allowing a maximum of six stack guards. Full control-flow protection is not included.
- Debugger Interaction: Faults that cause system resets must be tested without an active debugging session.
- Limited Evaluation: Testing was performed on only one development board, using one application workload and one or two trials per fault type.

CHAPTER 15
CONCLUSION AND FUTURE SCOPE

15.1 CONCLUSION

This project focuses on the design, implementation, and evaluation of an autonomous runtime firmware fault-localization framework using execution signatures on an STM32F446RE microcontroller running FreeRTOS. The primary objective is to identify abnormal firmware behaviour, localize potential faults, and initiate appropriate recovery actions without depending on external debugging equipment or continuous PC connectivity. The framework uses the hardware monitoring capabilities available within the ARM Cortex-M4 processor to observe firmware execution and analyse deviations from normal operating behaviour.

The framework captures eight categories of runtime information using on-chip hardware resources and lightweight software mechanisms. These include task switches, interrupt entries and exits, application stage transitions, peripheral errors, communication errors, execution timing information, memory-related faults, and watchdog events. The Data Watchpoint and Trace (DWT) cycle counter provides high-resolution execution timestamps, while FreeRTOS trace hooks collect task-level execution events. These events are stored in an 8 KB circular buffer to maintain a recent history of system execution without requiring large amounts of memory. Additionally, Memory Protection Unit (MPU) guards, processor fault status registers, peripheral configuration checksums, error flags, and the reset-cause register are used to identify different abnormal conditions. An independent watchdog provides additional protection against situations in which normal system execution cannot be maintained.

The framework generates a runtime execution signature at regular 100 ms intervals. During fault-free operation, the system learns a reference baseline representing the normal execution characteristics of the application. This baseline is retained on the target board so that subsequent execution behaviour can be compared against the learned reference, including after system resets. During normal operation, the monitor continuously evaluates task execution periods, execution durations, interrupt activity, application stage sequences, and peripheral status. Any significant deviation from the expected behaviour is analysed using predefined diagnostic rules to identify the potentially affected task, interrupt, or peripheral. A confidence score is assigned to indicate the likelihood that the identified component is associated with the detected abnormality.

When a fault is detected, the framework generates a diagnostic report containing the fault type, suspected component, confidence score, triggered diagnostic rules, program counter information where available, and the most recent 16 execution events. Recovery is performed using a predefined deterministic recovery table. If the same component experiences another failure within a 10-second interval, the framework escalates the recovery response. Fault reports are stored in flash memory for later retrieval, allowing diagnostic information to remain available even after a system reset. A command-line interface is also provided to inject and cancel faults, retrieve ground-truth information, inspect execution traces, and monitor the overall system status during testing.

The framework was evaluated using a soft real-time sensor application with ten injectable fault types. A real-time clock add-on was incorporated to measure the time associated with fault occurrence, detection, localization, and recovery. An 18-case experimental campaign was conducted on the STM32F446RE development board to assess the detection accuracy, localization capability, response time, and monitoring overhead. Out of the 18 test cases, all 15 fault events were detected, including two events that were not successfully generated by the fault injector. Fourteen cases were localized correctly on the first attempt, while the remaining case helped identify an implementation issue that was subsequently corrected and verified through repeated testing.

The measured runtime fault-reporting latency ranged from 31 to 172 ms, while memory-protection faults were trapped within 9 to 239 µs. No false alarms were observed during the clean-operation test, indicating that the monitoring mechanism could distinguish normal execution behaviour from the tested abnormal conditions. Furthermore, the monitor consumed approximately 1.2% of the available CPU processing capacity, demonstrating that continuous execution monitoring can be implemented with relatively low computational overhead.

Overall, the experimental results demonstrate that runtime execution signatures can provide a practical approach to autonomous firmware fault detection and localization on resource-constrained microcontrollers. The framework combines hardware-based execution monitoring, task-level behavioural analysis, confidence-based fault localization, deterministic recovery, and persistent diagnostic reporting within a single embedded platform. Its ability to operate directly on the target microcontroller without requiring external trace hardware makes it a useful approach for improving the dependability, maintainability, and diagnostic capabilities of real-time embedded systems.

15.2 FUTURE SCOPE

- Conduct 15–20 repeated tests for each fault type to obtain more reliable detection and false-alarm results.
- Improve the detection rules by increasing the variance limit or using separate rules for event-driven tasks to reduce false alarms.
- Compare the existing method with an Isolation Forest machine-learning model trained using recorded execution signatures.
- Test the framework on other Cortex-M microcontrollers, including those without an MPU, to identify the available monitoring features.
- Add application-level checks to detect incorrect data values that do not affect program execution.
- Test shorter monitoring intervals for applications with tighter timing requirements and measure their effect on detection speed and CPU usage.

REFERENCES

[1]A. Avizienis, J.-C. Laprie, B. Randell and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," IEEE Transactions on Dependable and Secure Computing, vol. 1, no. 1, pp. 11–33, Jan.–Mar. 2004.

[2] International Electrotechnical Commission, "IEC 61508: Functional safety of electrical/electronic/programmable electronic safety-related systems," 2nd ed., Geneva, 2010.

[3]International Organization for Standardization, "ISO 26262: Road vehicles – Functional safety," 2nd ed., Geneva, 2018.

[4] ARM Ltd., "ARMv7-M Architecture Reference Manual," ARM DDI 0403E, 2014.

[5]J. Yiu, The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors, 3rd ed. Oxford, U.K.: Newnes, 2014.

[6] STMicroelectronics, "RM0390 Reference manual: STM32F446xx advanced Arm-based 32-bit MCUs," Rev. 6, 2021.

[7]W. E. Wong, R. Gao, Y. Li, R. Abreu and F. Wotawa, "A survey on software fault localization," IEEE Transactions on Software Engineering, vol. 42, no. 8, pp. 707–740, Aug. 2016.

[8]B. Chang et al., "FirmRCA: Towards post-fuzzing analysis on ARM embedded firmware with efficient event-based fault localization," in Proc. IEEE Symposium on Security and Privacy (SP), 2025.

[9] A. Mahmood and E. J. McCluskey, "Concurrent error detection using watchdog processors – a survey," IEEE Transactions on Computers, vol. 37, no. 2, pp. 160–174, Feb. 1988.

[10] N. Oh, P. P. Shirvani and E. J. McCluskey, "Control-flow checking by software signatures," IEEE Transactions on Reliability, vol. 51, no. 1, pp. 111–122, Mar. 2002.

[11]J. A. Jones, M. J. Harrold and J. Stasko, "Visualization of test information to assist fault localization," in Proc. 24th International Conference on Software Engineering (ICSE), 2002, pp. 467–477.

[12]R. Abreu, P. Zoeteweij and A. J. C. van Gemund, "On the accuracy of spectrum-based fault localization," in Proc. Testing: Academic and Industrial Conference – Practice and Research Techniques (TAICPART-MUTATION), 2007, pp. 89–98.

[13]U. T. Gabor, "Software fault injection and localization in embedded systems," Ph.D. dissertation, Technische Universität Dortmund, Dortmund, Germany, 2021.

[14] ARM Ltd., "Cortex-M4 Technical Reference Manual," ARM DDI 0439B, 2010.

[15]M.-C. Hsueh, T. K. Tsai and R. K. Iyer, "Fault injection techniques and tools," Computer, vol. 30, no. 4, pp. 75–82, Apr. 1997.

[16]H. Ziade, R. Ayoubi and R. Velazco, "A survey on fault injection techniques," The International Arab Journal of Information Technology, vol. 1, no. 2, pp. 171–186, Jul. 2004.

[17] A. Benso and P. Prinetto, Eds., Fault Injection Techniques and Tools for Embedded Systems Reliability Evaluation. Boston, MA, USA: Kluwer Academic, 2003.

[18]Y. Li, S. Makar and S. Mitra, "CASP: Concurrent autonomous chip self-test using stored test patterns," in Proc. Design, Automation and Test in Europe (DATE), 2008, pp. 885–890.

[19]V. Chandola, A. Banerjee and V. Kumar, "Anomaly detection: A survey," ACM Computing Surveys, vol. 41, no. 3, pp. 15:1–15:58, Jul. 2009.

[20]F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation forest," in Proc. 8th IEEE International Conference on Data Mining (ICDM), 2008, pp. 413–422.

[21]B. P. Welford, "Note on a method for calculating corrected sums of squares and products," Technometrics, vol. 4, no. 3, pp. 419–420, Aug. 1962.

[22]R. Barry, Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide. Real Time Engineers Ltd., 2016.